



Eidgenössische Technische Hochschule Zürich  
Swiss Federal Institute of Technology Zurich



---

Spring Semester 2025

Prof. H.-A. Loeliger

Master Project

# On Constrained Convex Optimization Via Dual NUP Representations

Cyrill Jan Achermann

---

Advisor: Prof. Dr. Hans-Andrea Loeliger  
Co-Advisors: Hugo Aguetaz  
Yunpeng Li



# Acknowledgments

First, I would like to thank Professor H.-A. Loeliger, Yunpeng Li, and Hugo Aguetaz for the opportunity to undertake this project. The research area was interesting, and they fully met my wish to learn Rust in this project. Yunpeng's responses to my questions were constructive, and his expertise in the field was helpful. Finally, I would like to extend my special thanks to Hugo. His quick and straightforward help was welcoming, and our discussions were insightful and fun.

I really enjoyed this project at the ISI laboratory!

Zürich, 1 September 2025

Cyrill Achermann



# Abstract

Normals with unknown parameters (NUP) allow iteratively reweighted least-squares algorithms to solve constrained optimization problems. In this project, we compare two of these algorithms, namely, the *Iterative Reweighted Linear-Gaussian Estimation* (IRLGE) algorithm and the *Iterative Forward-Filtering Backward Dual Deciding* (IFFBDD) algorithm. We analyze their computational behavior and explore methods to minimize an  $L_1$  objective. We then extend one algorithm to state space models, including adaptations for handling input constraints and composite NUPs. We depict our algorithms in factor graphs, which is a good way to visualize their respective behavior. Finally, we provide a Rust implementation of the algorithms, along with remarks on the design choices we made in the implementations.

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                          | <b>1</b>  |
| <b>2</b> | <b>Constrained Optimization</b>                              | <b>3</b>  |
| 2.1      | Iterative Reweighted Linear-Gaussian Estimation . . . . .    | 4         |
| 2.1.1    | Problems . . . . .                                           | 6         |
| 2.2      | Iterative Forward-Filtering Backward Dual Deciding . . . . . | 6         |
| 2.2.1    | Idea . . . . .                                               | 6         |
| 2.2.2    | Simplification Work . . . . .                                | 8         |
| 2.2.3    | Main Algorithm . . . . .                                     | 13        |
| 2.2.4    | On The Internal NUP Parameter $\gamma$ . . . . .             | 15        |
| 2.3      | Comparing Algorithms . . . . .                               | 19        |
| 2.3.1    | On Minimizing $L_1$ Cost . . . . .                           | 19        |
| 2.3.2    | Iteration Scaling . . . . .                                  | 23        |
| 2.3.3    | Convergence Speed . . . . .                                  | 26        |
| <b>3</b> | <b>State Space Models</b>                                    | <b>30</b> |
| 3.1      | Enforcing Constraints on Input . . . . .                     | 32        |
| 3.2      | Composite NUP Priors . . . . .                               | 33        |
| 3.3      | Example And Limitations . . . . .                            | 35        |
| <b>4</b> | <b>Implementation</b>                                        | <b>38</b> |
| 4.1      | General Structure . . . . .                                  | 38        |
| 4.2      | Enumerations . . . . .                                       | 39        |
| 4.3      | Reference Counting . . . . .                                 | 41        |
| 4.4      | BLAS . . . . .                                               | 41        |
| 4.5      | Traits . . . . .                                             | 43        |
| 4.6      | Learnings . . . . .                                          | 44        |
| <b>5</b> | <b>Conclusions And Future Work</b>                           | <b>46</b> |
|          | <b>Bibliography</b>                                          | <b>48</b> |
|          | <b>List of Figures</b>                                       | <b>49</b> |
|          | <b>List of Tables</b>                                        | <b>49</b> |

# Chapter 1

## Introduction

Normals with unknown parameters (NUP) allow iteratively reweighted least-squares algorithms to solve constrained optimization problems. This project aims to explore further the algorithm proposed in [1]. We will compare it to existing NUP proposals, such as in [2]. Additionally, we show one adaptation to state space models. Furthermore, this project provides practical Rust implementations of versions of algorithms proposed in [1].

The report is structured as follows. In Chapter 1, we provide background information on factor graphs and NUP representations. Chapter 2 introduces the main objective, the relevant algorithms, and explores their behavior. In Chapter 3, we extend an algorithm to the state space model setting and investigate extensions. Before finishing the report in Chapter 5, we give some notes on the Rust implementation of the different algorithms.

In this master's project, we use factor graphs to represent the proposed algorithms. A factor graph is a diagram that represents the factorization of a function of several variables [3, Section 1.2]. For example,

$$f(u, w, x, y, z) = f_1(u, w, x)f_2(x, y, z)f_3(z), \quad (1.1)$$

is decomposed into three functions, where the variables  $u$ ,  $w$ ,  $x$ ,  $y$ , and  $z$  represent edges, and the functions represent the nodes. Figure 2.2 depicts a factor graph we will use later. Using a factor graph, we can pass *messages*, e.g.,  $\vec{\mu}_X$ , along the edges of the factor graph and the nodes. The nodes transform the messages, depending on their functionality. The arrows of a message  $\vec{\mu}_X$  indicate the direction along which a message travels. The messages can take many forms; however, in our project, the messages  $\vec{\mu}_{X_i}$  will be multivariate Gaussian messages, i.e.

$$\vec{\mu}_{X_i}(x_i) = \mathcal{N}(x_i; \vec{m}_{X_i}, \vec{V}_{X_i}) \propto e^{-\frac{1}{2}(x_i - \vec{m}_{X_i})^\top \vec{V}_{X_i}^{-1}(x_i - \vec{m}_{X_i})}. \quad (1.2)$$

Here,  $\vec{m}_{X_i} \in \mathbb{R}^k$  is the mean vector and  $\vec{V}_{X_i} \in \mathbb{R}^{k \times k}$  is the covariance matrix of the multivariate Gaussian message  $\vec{\mu}_{X_i}$ . For Gaussian messages, we have tabulated the message-passing rules for various node types. Consider further [3, Chapter 3]. [3, Table 3.2 and Table 3.3] provide the essential update rules, which are *sum-product message-passing* rules.

The key part is now, if  $f(x_1, \dots, x_n)$  can be represented by a cycle-free factor graph and has a unique maximizer, then computing this maximizer

$$(\hat{x}_1, \dots, \hat{x}_n) \triangleq \underset{x_1, \dots, x_n}{\operatorname{argmax}} f(x_1, \dots, x_n) \quad (1.3)$$

can be expressed by messages  $\overrightarrow{\mu}_{X_i}$  and  $\overleftarrow{\mu}_{X_i}$  following the *max-product message computation rule*, consider further [3, Section 1.7]. Importantly for Gaussian messages, the *sum-product message-passing* rules referenced above coincide with the *max-product message computation rule*. We can use factor graphs to graphically represent optimization algorithms by describing the message-passing schedule. If we want to minimize a sum of cost functions  $\kappa_i : \mathbb{R}^k \rightarrow \mathbb{R}_{\geq 0}$ , we can achieve this by calculating

$$\hat{x} \triangleq \underset{x}{\operatorname{argmin}} \sum_i \kappa_i(x) \quad (1.4)$$

$$= \underset{x}{\operatorname{argmax}} \prod_i e^{-\kappa_i(x)}. \quad (1.5)$$

The term  $e^{-\kappa_i(x)}$  is not necessarily Gaussian, but we can represent this cost as a Gaussian message by a *Normal with Unknown Parameters* (NUP) if we can find a  $g_i(\theta_i)$ , such that

$$e^{-\kappa_i(x)} = \max_{\theta_i} e^{-\frac{1}{2}Q(x;\theta_i)} g_i(\theta_i) \quad (1.6)$$

where  $Q(x; \theta_i)$  is a quadratic expression of the form

$$Q(x; \theta_i) = (x - m)^\top V^{-1} (x - m) + \text{const.}, \quad (1.7)$$

and the  $\theta_i$  parameter subsumes the mean and the covariance. Then we can rewrite Equation (1.5) to

$$\hat{x} = \underset{x}{\operatorname{argmax}} \prod_i e^{-\kappa_i(x)} \quad (1.8)$$

$$= \underset{x}{\operatorname{argmax}} \prod_i \max_{\theta_i} e^{-\frac{1}{2}Q(x;\theta_i)} g_i(\theta_i) \quad (1.9)$$

$$= \underset{x}{\operatorname{argmax}} \max_{\theta_1, \dots, \theta_n} \underbrace{\prod_i e^{-\frac{1}{2}Q(x;\theta_i)} g_i(\theta_i)}_{=f(x, \theta_1, \dots, \theta_n)}. \quad (1.10)$$

[3, Chapter 6] provides a more detailed approach. If the costs in Equation (1.5) are strictly convex, i.e., there exists a unique solution to the problem, we can solve this double maximization problem by alternating until convergence between

1. for fixed  $\theta_i$ , calculate  $\operatorname{argmax}_x f(x, \theta_1, \dots, \theta_n)$ ,
2. for fixed  $x$ , calculate  $\operatorname{argmax}_{\theta_1, \dots, \theta_n} f(x, \theta_1, \dots, \theta_n)$ . Note that all parameters are decoupled, and we get simpler maximizations, for which usually closed-form expressions exist.

With this introduction, we can continue with the formally stated problem and describe the optimization problem we want to solve.



## Chapter 2

# Constrained Optimization

We encounter optimization problems in many real-world tasks. It involves finding the best "strategy" for a certain system, such as minimizing fuel consumption in a car or maximizing production output of a factory. In practice, these systems often have restrictions, such as a maximum duration of a work-shift. We can express such restrictions as constraints on the optimization problem. Constraint optimization allows us to find solutions that are not only optimal with respect to our objective but also feasible with respect to the problem's requirements.

In this chapter, we provide algorithms that can solve convex constrained optimization problems.

We can formulate our objective as

$$\begin{aligned} & \text{minimize } Q(x) \\ & \text{subject to linear constraints,} \end{aligned} \tag{2.1}$$

where  $Q(x)$  is a quadratic expression,  $x \in \mathbb{R}^k$  and the linear constraints are given in the form

$$Cx \leq b, \tag{2.2}$$

where  $\leq$  should be understood element-wise,  $C \in \mathbb{R}^{n \times k}$ ,  $b \in \mathbb{R}^n$ . Throughout this project, we additionally assume that the problem is feasible.<sup>1</sup>

Please note that we can construct the other constraints, i.e.,  $\geq$ ,  $=$ , and  $\leq \cdot \geq$ , using just this inequality given in Equation (2.2). For example, an equality constraint  $C_{i,\cdot}x = b_i$ , where  $b_i \in \mathbb{R}$  and  $C_{i,\cdot}$  is a row vector of  $C$ , can be constructed by

$$\begin{bmatrix} C_{i,\cdot} \\ -C_{i,\cdot} \end{bmatrix} x \leq \begin{bmatrix} b_i \\ -b_i \end{bmatrix} \tag{2.3}$$

where the intersection of the feasible domains results in  $C_{i,\cdot}x = b_i$ . This example shows that we do not lose generality in terms of possible constraints when restricting ourselves to " $\leq$ " constraints. From now on, we will thus stick to the notation  $Cx \leq b$ , with the appropriate dimension.

---

<sup>1</sup>To always generate a feasible problem, we generate  $C$ , and  $x$  at random, then calculate  $Cx = b$ . Then we know that there must exist a solution  $x$ , no matter the constraints.

**Algorithm 1** IRLGE

---

```

1: Initialize  $x = 0$  and  $\beta_i = 1e - 5$  for  $i = 1, \dots, n$ 
2: while constraints are not satisfied do
3:   while not converged do
4:     Update upward messages:
5:     for  $i = 1$  to  $n$  do
6:        $\hat{y}_i = C_{i,\cdot} \hat{x}$ 
7:        $\overleftarrow{V}_{Y_i} = \frac{|\hat{y}_i - b_i|}{\beta_i}$ 
8:        $\overleftarrow{m}_{Y_i} = (b_i - |\hat{y}_i - b_i|)$ 
9:     end for
10:    Standard forward Kalman filtering:
11:    Set  $\overrightarrow{m}_{X_1}$  and  $\overrightarrow{V}_{X_1}$  according to  $\overrightarrow{\mu}_{X_1}$ 
12:    for  $i = 1$  to  $n$  do
13:       $G_i = (\overleftarrow{V}_{Y_i} + C_{i,\cdot} \overrightarrow{V}_{X_i} C_{i,\cdot}^\top)^{-1}$ 
14:       $\overrightarrow{V}_{X_{i+1}} = \overrightarrow{V}_{X_i} - \overrightarrow{V}_{X_i} C_{i,\cdot}^\top G_i C_{i,\cdot} \overrightarrow{V}_{X_i}$ 
15:       $\overrightarrow{m}_{X_{i+1}} = \overrightarrow{m}_{X_i} + \overrightarrow{V}_{X_i} C_{i,\cdot}^\top G_i (\overleftarrow{m}_{Y_i} - C_{i,\cdot} \overrightarrow{m}_{X_i})$ 
16:    end for
17:     $\hat{x} = \overrightarrow{m}_{X_{n+1}}$ 
18:    Update  $\overrightarrow{\mu}_{X_1}$  according to the modeled prior.
19:  end while
20:  Check constraints and update  $\beta$ 's
21:  for  $i = 1$  to  $n$  do
22:    if  $C_{i,\cdot} \hat{x} > b_i$  then
23:       $\beta_i = 2 \cdot \beta_i$ 
24:    end if
25:  end for
26: end while

```

---

## 2.1 Iterative Reweighted Linear-Gaussian Estimation

We can solve the objective given in Equation (2.1) with the iterative algorithm, called *Iterative Reweighted Linear-Gaussian Estimation* (IRLGE). We outline the pseudo-code in Algorithm 1. Figure 2.2 provides the factor graph for Algorithm 1. The box labeled  $\overrightarrow{\mu}_{X_1}$  is a different NUP box, which models the quadratic cost of Equation (2.1) and can, in our case, either be an  $L_1$  or an  $L_2$  cost. NUP<sub>*i*</sub> attempts to enforce the *i*-th constraint by providing additional cost  $\kappa_i$ . Please note that Line 13 in Algorithm 1 corresponds to a scalar inversion and not a matrix inversion. Matrix inversions are problematic in terms of stability and computational cost. Additionally, Line 14 is a rank-one update. Work in [2] provides the NUP update rules in Line 7 and Line 8. Figure 2.1 illustrates an exemplary cost, with which we can try to make sense of the NUP update rules.

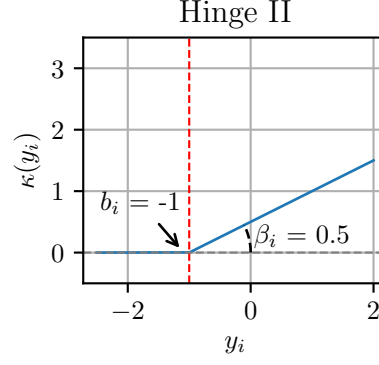


Figure 2.1: Visualization of the hinge II cost function.

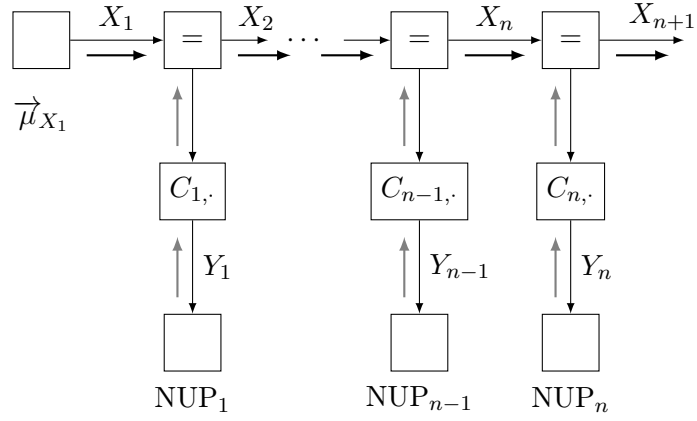


Figure 2.2: Factor graph to model constrained optimization.

- If  $\hat{y}_i \leq b_i$ , the constraint is satisfied, we set the backward mean  $\overleftarrow{m}_{Y_i}$  to be  $\hat{y}_i$ . The further away we are from the border  $b_i$ , the more uncertainty we allow.
- If  $\hat{y}_i > b_i$ , the constraint is violated, and we push back the next mean  $\overleftarrow{m}_{Y_i}$  by  $2 \cdot b_i$ , conveying the same uncertainty rule as before.

The rules above make intuitive sense for handling an iterative algorithm that should satisfy constraints like these. By using these costs, we relax the constraint optimization problem in Equation (2.1) to an unconstrained problem with additional costs. The larger the slope  $\beta_i$ , the closer its NUP resembles a constraint. Furthermore, a hinge II cost function represents a constraint  $y_i \leq b_i$  if  $\beta_i \rightarrow \infty$ .

### 2.1.1 Problems

Algorithm 1 works well for some simpler setups. However, we can observe some problems in this algorithm:

1. The free slope parameter  $\beta_i$  is difficult to choose. The larger  $\beta_i$ , the slower the inner loop converges. However, we need to increase  $\beta_i$  to satisfy the violated constraints. Algorithm 1 doubles  $\beta_i$  if the constraint is not satisfied. This is a practical solution but it might not be optimal. Additionally, we cannot choose  $\beta_i = \infty$ , as this would result in  $\overleftarrow{V}_{Y_i} = 0$ , which is problematic.
2. If  $\hat{y}_i = b_i$ , we also get  $\overleftarrow{V}_{Y_i} = 0$  and the algorithm is stuck.
3. Since we have an outer loop and an inner loop, this algorithm is generally slow. Indeed, for larger problems, in terms of  $x \in \mathbb{R}^k$ , where  $k$  is large or for many constraints, the algorithm starts to become slow.

## 2.2 Iterative Forward-Filtering Backward Dual Deciding

Exploring this algorithm is the central part of the project. It was derived by Li et al. [1] as an extension and improvement of [4] and [2].

### 2.2.1 Idea

The main idea is to use the Legendre transform to dualize some variables of a factor graph, especially variables connected to the costs  $\kappa$ , which try to model the constraints. The Legendre transform is given by

$$\kappa^*(\tilde{y}) = \sup_y \tilde{y}y - \kappa(y). \quad (2.4)$$

When we apply the Legendre transform to variables of a factor graph, we can think of *dualizing* these variables. Just as in Algorithm 1, we want to represent the *dual cost*  $\kappa^*(\tilde{y})$  with an NUP, i.e.,

$$e^{-\kappa^*(\tilde{y})} = \sup_{\tilde{\theta}} e^{\frac{1}{2}Q(\tilde{y};\tilde{\theta})}\tilde{g}(\tilde{\theta}), \quad (2.5)$$

where  $Q(\tilde{y};\tilde{\theta})$  is a quadratic expression in  $\tilde{y}$  and  $\tilde{g}$  is a suitable function. For further information, consider [1, Section III.C]. To transform an incoming primal scalar Gaussian message to its dual, we perform

$$\vec{\mu}_{\tilde{Y}}(\tilde{y}) \propto \max_y \vec{\mu}_Y(y) e^{-\tilde{y}y}. \quad (2.6)$$

The relation between the primal and the dual Gaussian message is given by

$$\vec{m}_{\tilde{Y}} = \vec{\xi}_Y \quad \text{and} \quad \vec{V}_{\tilde{Y}} = \vec{W}_Y \quad (2.7)$$

$$\vec{\xi}_{\tilde{Y}} = \vec{m}_Y \quad \text{and} \quad \vec{W}_{\tilde{Y}} = \vec{V}_Y \quad (2.8)$$

$$\overleftarrow{m}_{\tilde{Y}} = -\overleftarrow{\xi}_Y \quad \text{and} \quad \overleftarrow{V}_{\tilde{Y}} = \overleftarrow{W}_Y \quad (2.9)$$

$$\overleftarrow{\xi}_{\tilde{Y}} = -\overleftarrow{m}_Y \quad \text{and} \quad \overleftarrow{W}_{\tilde{Y}} = \overleftarrow{V}_Y. \quad (2.10)$$

By [1, Theorem 1], dualizing the cost and some Gaussian messages can be done, and the objectives, which the respective factor graphs try to solve, coincide. The idea is then to iteratively *dual decide* dual variables connected to the dual cost  $\kappa^*$  and pass this information back into the factor graph. Dual deciding corresponds to

$$\hat{\tilde{y}} = \underset{\tilde{y}}{\operatorname{argmin}} \vec{\mu}_{\tilde{Y}}(\tilde{y}) e^{\kappa^*(\tilde{y})}. \quad (2.11)$$

Fixing a dual variable  $\tilde{Y} = \hat{\tilde{y}}$  can be expressed by the marginal mean  $m_{\tilde{Y}} = \hat{\tilde{y}}$  and covariance  $V_{\tilde{Y}} = 0$ . We then transform this dual decided variable  $\tilde{Y}$  back to get a Gaussian message in the primal domain. With some reformulations, we get the primal message as

$$\overleftarrow{\xi}_Y = -\hat{\tilde{y}} \quad \text{and} \quad \overleftarrow{W}_Y = 0. \quad (2.12)$$

After dual deciding, we also need to update the NUP based on  $\hat{\tilde{y}}$ . This results in a Gaussian message different from the dual decision value. We will see the exact update rules later in Section 2.2.2.

For our purposes, however, the whole part above can be summarized by the following:

1. For an incoming message  $\vec{\mu}_Y$  into an NUP box, we dual decide  $\hat{\tilde{y}}$  based on a table. Dual deciding results in a Gaussian message  $\overleftarrow{\xi}_Y = \hat{\tilde{y}}$  and  $\overleftarrow{W}_Y = 0$ .
2. After dual deciding, we update internal NUP parameters to obtain a new Gaussian message  $\overleftarrow{\mu}_Y$ , which differs from the dual-decision above.

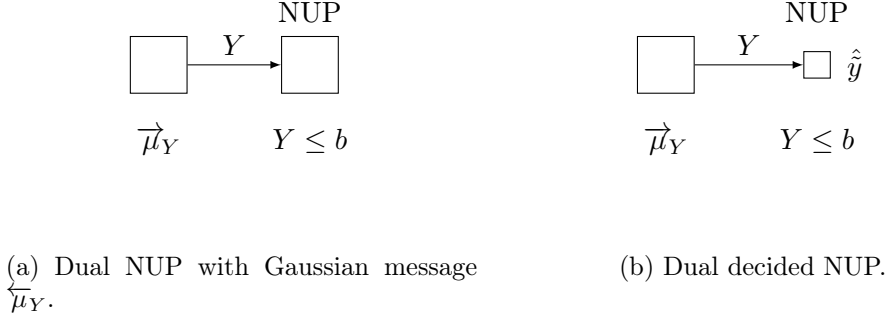


Figure 2.3: Dual deciding a NUP.

We need to distinguish carefully whether we are in a *dual deciding* phase or not, because both the dual decided message and the Gaussian message for fixed internal NUP parameters originate from the same NUP box; however, they occur in different *phases* of the algorithm. We will see when to use what in Algorithm 2. Figure 2.3a and Figure 2.3b show the steps above in a factor graph.

Figure 2.3a indicates fixed internal NUP parameters; with fixed internal parameters, the NUP admits a Gaussian message. This *phase* corresponds to step 2 in the list above.

Figure 2.3b indicates a *dual decided* variable with a resulting backward Gaussian message as in Equation (2.12). This *phase* corresponds to step 1 in the list above. Please note that this small white box is not a standard factor graph notation. In this project, we visually indicate which NUP boxes have already been *dual decided* by drawing them as small white boxes. Moreover, it is not a filled black box, because, unlike a black box, which indicates a fixed variable, this white box indicates zero precision.

### 2.2.2 Simplification Work

For some important costs  $\kappa$ , the *dual decide* update rules and their NUP representation have been worked out in terms of the dual variable  $\tilde{Y}$  in [1, Table I and Table III]. With the relations from Equation (2.7) to Equation (2.10), we can express the update rules just in terms of the primal variable  $Y$ . Additionally, note that the following dual costs  $\kappa^*$  cannot be represented directly by an NUP but need a proxy  $\check{\kappa}$  with an additional parameter  $\gamma$ . However, by adjusting  $\gamma$ , we get that the dual cost  $\kappa^*$  and the proxy cost  $\check{\kappa}$  coincide in the relevant parts.

Table 2.1 illustrates the updated rules for the costs  $\kappa$ , expressed in the primal variables. Remember that decided  $\hat{y}$  also corresponds to  $\overleftarrow{\xi}_Y = \hat{y}$  and  $\overleftarrow{W}_Y = 0$ . Thus, the *dual decided* message does not coincide with the Gaussian message output from the dual NUP box after updating the internal parameters based on  $\hat{y}$ .

| Cost                                                                                                 | Case                                                 | $\gamma$                                            | $\hat{y}$                   | $\xi_Y$                                                                                                     | $\overleftarrow{W}_Y$                                    |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| $\beta y - c $                                                                                       | $(\vec{m}_Y - c)/\vec{V}_Y < -\beta$                 | $\max\{\gamma, c - \vec{m}_Y - \vec{V}_Y\beta\}$    | $-\beta$                    | $\beta$                                                                                                     | 0                                                        |
|                                                                                                      | $-\beta \leq (\vec{m}_Y - c)/\vec{V}_Y \leq \beta$   | no update                                           | $(\vec{m}_Y - c)/\vec{V}_Y$ | $\frac{a \hat{y} + \beta  \hat{y} - \beta }{\gamma\beta} + \frac{ \hat{y} - \beta  -  \hat{y} + \beta }{2}$ | $\frac{ \hat{y} + \beta  \hat{y} - \beta }{\gamma\beta}$ |
|                                                                                                      | $(\vec{m}_Y - c)/\vec{V}_Y > \beta$                  | $\max\{\gamma, \vec{m}_Y - c - \vec{V}_Y\beta\}$    | $\beta$                     | $-\beta$                                                                                                    | 0                                                        |
| $\begin{cases} \beta(a - y), & y < a \\ 0, & y \geq a \end{cases}$                                   | $(\vec{m}_Y - a)/\vec{V}_Y < -\beta$                 | $\max\{\gamma, a - \vec{m}_Y - \vec{V}_Y\beta\}$    | $-\beta$                    | $\beta$                                                                                                     | 0                                                        |
|                                                                                                      | $-\beta \leq (\vec{m}_Y - a)/\vec{V}_Y \leq 0$       | no update                                           | $(\vec{m}_Y - a)/\vec{V}_Y$ | $\frac{2a \hat{y} + \beta  \hat{y} }{\gamma\beta} +  \hat{y} $                                              | $\frac{2 \hat{y} + \beta  \hat{y} }{\gamma\beta}$        |
|                                                                                                      | $(\vec{m}_Y - a)/\vec{V}_Y > 0$                      | $\max\{\gamma, \vec{m}_Y - a\}$                     | 0                           | 0                                                                                                           | 0                                                        |
| $\begin{cases} 0, & y \leq b \\ \beta(y - b), & y > b \end{cases}$                                   | $(\vec{m}_Y - b)/\vec{V}_Y < 0$                      | $\max\{\gamma, b - \vec{m}_Y\}$                     | 0                           | 0                                                                                                           | 0                                                        |
|                                                                                                      | $0 \leq (\vec{m}_Y - b)/\vec{V}_Y \leq \beta$        | no update                                           | $(\vec{m}_Y - b)/\vec{V}_Y$ | $\frac{2b \hat{y}  \hat{y} - \beta }{\gamma\beta} -  \hat{y} $                                              | $\frac{2 \hat{y}  \hat{y} - \beta }{\gamma\beta}$        |
|                                                                                                      | $(\vec{m}_Y - b)/\vec{V}_Y > \beta$                  | $\max\{\gamma, \vec{m}_Y - b - \vec{V}_Y\beta\}$    | $\beta$                     | $-\beta$                                                                                                    | 0                                                        |
| $\begin{cases} 2\beta(a - y), & y < a \\ 0, & a \leq y \leq b \\ 2\beta(y - b), & y > b \end{cases}$ | $(\vec{m}_Y - a)/\vec{V}_Y < -2\beta$                | $\max\{\gamma_a, a - \vec{m}_Y - 2\vec{V}_Y\beta\}$ | $-2\beta$                   | $2\beta$                                                                                                    | 0                                                        |
|                                                                                                      | $-2\beta \leq (\vec{m}_Y - a)/\vec{V}_Y < 0$         | $\max\{\gamma_b, b - \vec{m}_Y\}$                   | $(\vec{m}_Y - a)/\vec{V}_Y$ | $\frac{a \hat{y} + 2\beta  \hat{y} }{\gamma_a\beta} +  \hat{y} $                                            | $\frac{ \hat{y} + 2\beta  \hat{y} }{\gamma_a\beta}$      |
|                                                                                                      | $a/\vec{V}_Y \leq \vec{m}_Y/\vec{V}_Y < b/\vec{V}_Y$ | no update                                           | 0                           | 0                                                                                                           | 0                                                        |
|                                                                                                      | $0 \leq (\vec{m}_Y - b)/\vec{V}_Y < 2\beta$          | $\max\{\gamma_a, \vec{m}_Y - a\}$                   | $(\vec{m}_Y - b)/\vec{V}_Y$ | $\frac{b \hat{y} - 2\beta  \hat{y} }{\gamma_b\beta} -  \hat{y} $                                            | $\frac{ \hat{y}  \hat{y} - 2\beta }{\gamma_b\beta}$      |
|                                                                                                      | $2\beta \leq (\vec{m}_Y - b)/\vec{V}_Y$              | $\max\{\gamma_b, \vec{m}_Y - b - 2\vec{V}_Y\beta\}$ | $2\beta$                    | $-2\beta$                                                                                                   | 0                                                        |
|                                                                                                      |                                                      | $\max\{\gamma_a, \vec{m}_Y - a\}$                   |                             |                                                                                                             |                                                          |

Table 2.1: Dual variable update rules for all dual costs with manual  $\gamma$ .

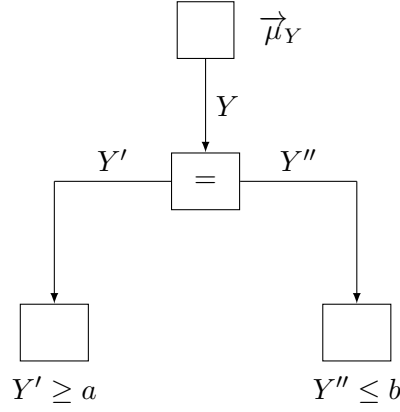


Figure 2.4: Update rules for combined NUP to model interval constraint.

We can model constraints and not only costs by letting the slope  $\beta$  go to infinity, formally,  $\beta \rightarrow +\infty$ ; [1, Appendix B] derived this method. By taking the limit in the cells of Table 2.1, we get a new table for constraints.

Please note that for both Table 2.1 and Table 2.2, we derived the Vapnik cost and the interval constraint by inspecting the factor graph in Figure 2.4. Thus, we model an interval constraint by combining two half-space constraints. In the end, we aim to decide on both and obtain the backward message  $\overleftarrow{\mu}_Y$  for the next forward filter pass in terms of  $\overrightarrow{\mu}_Y$ ,  $\overleftarrow{\mu}_{Y'}$ , and  $\overleftarrow{\mu}_{Y''}$ , based on the previous backward decision. We can safely assume that one of the two previous messages  $\overleftarrow{\mu}_{Y'}$  or  $\overleftarrow{\mu}_{Y''}$  was zero, as long as  $a \leq b$ , because we cannot violate both constraints simultaneously. Without loss of generality, assume that  $\overleftarrow{\xi}_{Y'} = 0$  and  $\overleftarrow{W}_{Y'} = 0$  from the previous iteration. Thus  $\overrightarrow{\xi}_{Y''} = \overrightarrow{\xi}_Y$  and  $\overrightarrow{W}_{Y''} = \overrightarrow{W}_Y$ .

We now have to consider three cases:

- a)  $\overrightarrow{m}_{Y''} = \overrightarrow{m}_Y > b$ , yielding the decision  $\hat{y}'' = (\overrightarrow{m}_Y - b)/\overrightarrow{V}_Y$  and the backward messages  $\overleftarrow{\xi}_{Y''} = (b/\gamma_b - 1)\hat{y}''$  and  $\overleftarrow{W}_{Y''} = |\hat{y}''|/\gamma_b$ . Combining the decision with the incoming message  $\overrightarrow{\mu}_Y$  at the equality node, we get

$$\overrightarrow{W}_{Y'} = \overrightarrow{W}_Y + \overleftarrow{W}_{Y''} \quad (2.13)$$

$$= \overrightarrow{W}_Y, \quad (2.14)$$

$$\overrightarrow{\xi}_{Y'} = \overrightarrow{\xi}_Y + \overleftarrow{\xi}_{Y''} \quad (2.15)$$

$$= \overrightarrow{W}_Y \overrightarrow{m}_Y - (\overrightarrow{m}_Y - b) \overrightarrow{W}_Y \quad (2.16)$$

$$= \overrightarrow{W}_Y b, \quad (2.17)$$

$$\overrightarrow{m}_{Y'} = b. \quad (2.18)$$



| Constraint        | Case                      | $\gamma$                              | $\hat{\vec{y}}$             | $\overleftarrow{\xi}_Y$           | $\overleftarrow{W}_Y$      |
|-------------------|---------------------------|---------------------------------------|-----------------------------|-----------------------------------|----------------------------|
| $Y = c$           | $\vec{m}_Y < c$           | $\max\{\gamma_b, (c - \vec{m}_Y)/2\}$ | $(\vec{m}_Y - c)/\vec{V}_Y$ | $(1 + c/\gamma_a) \hat{\vec{y}} $ | $ \hat{\vec{y}} /\gamma_a$ |
|                   | $\vec{m}_Y \geq c$        | $\max\{\gamma_a, (c - \vec{m}_Y)/2\}$ |                             | $(c/\gamma_b - 1) \hat{\vec{y}} $ | $ \hat{\vec{y}} /\gamma_b$ |
| $Y \geq a$        | $\vec{m}_Y \geq a$        | $\max\{\gamma, (\vec{m}_Y - a)/2\}$   | 0                           | 0                                 | 0                          |
|                   | $\vec{m}_Y < a$           | no update                             | $(\vec{m}_Y - a)/\vec{V}_Y$ | $(1 + a/\gamma) \hat{\vec{y}} $   | $ \hat{\vec{y}} /\gamma$   |
| $Y \leq b$        | $\vec{m}_Y \leq b$        | $\max\{\gamma, (b - \vec{m}_Y)/2\}$   | 0                           | 0                                 | 0                          |
|                   | $\vec{m}_Y > b$           | no update                             | $(\vec{m}_Y - b)/\vec{V}_Y$ | $(b/\gamma - 1) \hat{\vec{y}} $   | $ \hat{\vec{y}} /\gamma$   |
| $a \leq Y \leq b$ | $a \leq \vec{m}_Y \leq b$ | $\max\{\gamma_a, (\vec{m}_Y - a)/2\}$ | 0                           | 0                                 | 0                          |
|                   |                           | $\max\{\gamma_b, (b - \vec{m}_Y)/2\}$ |                             |                                   |                            |
|                   | $\vec{m}_Y < a$           | $\max\{\gamma_b, (b - \vec{m}_Y)/2\}$ | $(\vec{m}_Y - a)/\vec{V}_Y$ | $(1 + a/\gamma_a) \hat{\vec{y}} $ | $ \hat{\vec{y}} /\gamma_a$ |
|                   | $\vec{m}_Y > b$           | $\max\{\gamma_a, (\vec{m}_Y - a)/2\}$ | $(\vec{m}_Y - b)/\vec{V}_Y$ | $(b/\gamma_b - 1) \hat{\vec{y}} $ | $ \hat{\vec{y}} /\gamma_b$ |

Table 2.2: Summarized update rules for all dual constraints.

Since  $\vec{m}_{Y'} = b \geq a$ , the constraint  $Y' \geq a$  is satisfied and we get  $\hat{y}' = 0$ ,  $\overleftarrow{W}_{Y'} = 0$  and  $\overleftarrow{\xi}_{Y'} = 0$ . Obtaining

$$\hat{y} = \hat{y}' + \hat{y}'' \quad (2.19)$$

$$= \frac{\vec{m}_Y - b}{\vec{V}_Y}. \quad (2.20)$$

We obtain the message for the forward filter pass by:

$$\overleftarrow{\xi}_Y = \overleftarrow{\xi}_{Y'} + \overleftarrow{\xi}_{Y''} \quad (2.21)$$

$$= \left(\frac{b}{\gamma_b} - 1\right)|\hat{y}''|, \quad (2.22)$$

$$\overleftarrow{W}_Y = \overleftarrow{W}_{Y'} + \overleftarrow{W}_{Y''} \quad (2.23)$$

$$= \frac{|\hat{y}''|}{\gamma_b}. \quad (2.24)$$

- b)  $a \leq \vec{m}_{Y''} = \vec{m}_Y \leq b$ , yielding the decision  $\hat{y}'' = 0$  and the backward messages  $\overleftarrow{\xi}_{Y''} = 0$  and  $\overleftarrow{W}_{Y''} = 0$ .

Getting:

$$\overrightarrow{\xi}_{Y'} = \overrightarrow{\xi}_Y - \hat{y}'' \quad (2.25)$$

$$= \overrightarrow{\xi}_Y, \quad (2.26)$$

$$\overrightarrow{W}_{Y'} = \overrightarrow{W}_Y + 0 \quad (2.27)$$

$$= \overrightarrow{W}_Y. \quad (2.28)$$

Similarly, we get  $\hat{y}' = 0$  and the backward messages  $\overleftarrow{\xi}_{Y'} = 0$  and  $\overleftarrow{W}_{Y'} = 0$ , thus

$$\hat{y} = \hat{y}' + \hat{y}'' \quad (2.29)$$

$$= 0, \quad (2.30)$$

$$\overleftarrow{\xi}_Y = \overleftarrow{\xi}_{Y'} + \overleftarrow{\xi}_{Y''} \quad (2.31)$$

$$= 0, \quad (2.32)$$

$$\overleftarrow{W}_Y = \overleftarrow{W}_{Y'} + \overleftarrow{W}_{Y''} \quad (2.33)$$

$$= 0. \quad (2.34)$$

- c)  $a > \vec{m}_{Y''} = \vec{m}_Y$ , yielding the decision  $\hat{y}'' = 0$  and the backward messages  $\overleftarrow{\xi}_{Y''} = 0$  and  $\overleftarrow{W}_{Y''} = 0$ .

We have:

$$\overrightarrow{\xi}_{Y'} = \overrightarrow{\xi}_Y, \quad (2.35)$$

$$\overrightarrow{W}_{Y'} = \overrightarrow{W}_Y. \quad (2.36)$$

| $\kappa(y)$    | $\kappa^*(\tilde{y})$                                                                                                                    | $\check{\kappa}(\tilde{y})$                                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\beta y - c $ | $\begin{cases} +\infty & \tilde{y} < -\beta \\ c\tilde{y} & -\beta \leq \tilde{y} \leq \beta \\ +\infty & \tilde{y} > \beta \end{cases}$ | $\begin{cases} (c - \gamma)\tilde{y} - \beta\gamma & \tilde{y} < -\beta \\ c\tilde{y} & -\beta \leq \tilde{y} \leq \beta \\ (c + \gamma)\tilde{y} - \beta\gamma & \tilde{y} > \beta \end{cases}$ |

Table 2.3: The  $L_1$  cost function  $\kappa$ , its dual  $\kappa^*$  and its dual proxy  $\check{\kappa}$  taken from [1, Table I and Table II].

This results in the decision  $\hat{y}' = (\vec{m}_Y - a)/\vec{V}_Y$  and the backward messages  $\overleftarrow{\xi}_{Y'} = (1 + a/\gamma_a)|\hat{y}'|$  and  $\overleftarrow{W}_{Y'} = |\hat{y}'|/\gamma_a$ . Finally

$$\hat{y} = \hat{y}' + \hat{y}'' \quad (2.37)$$

$$= \frac{\vec{m}_Y - a}{\vec{V}_Y}, \quad (2.38)$$

$$\overleftarrow{\xi}_Y = \overleftarrow{\xi}_{Y'} + \overleftarrow{\xi}_{Y''} \quad (2.39)$$

$$= (1 + \frac{a}{\gamma_a})|\hat{y}'|, \quad (2.40)$$

$$\overleftarrow{W}_Y = \overleftarrow{W}_{Y'} + \overleftarrow{W}_{Y''} \quad (2.41)$$

$$= \frac{|\hat{y}'|}{\gamma_a}. \quad (2.42)$$

We can carry out a similar example to get the update rules for the Vapnik cost in Table 2.1. By setting the slope parameter to  $2\beta$ , we get the same update rules as in [1], except that we have two  $\gamma$ . It remains to be tested how the two methods influence the behavior of Algorithm 2. Additionally, we can just set  $a = b = c$  to get an equality constraint. This derivation also shows that after the dual decision, the posterior of  $\vec{\mu}_Y$  and the dual decision will always satisfy the constraint.

### 2.2.3 Main Algorithm

Having the building blocks and the simplified tables, we can now present the main algorithm called *Iterative Forward-Filtering Backward Dual Deciding* (IFFBDD). Remember that we want to solve objectives of the form of Equation (2.1); thus, we currently do not need the dual NUP representation for costs. Algorithm 2 presents the pseudo-code, which we adapted from [1, Algorithm 2] but only with primal messages.

Notice the first phase, i.e., the *Kalman filtering* step, is the same as for Algorithm 1, but the NUPs are different. We provide a visual representation of Algorithm 2 in Figure 2.2 and Figure 2.5.

**Algorithm 2** IFFBDD

---

```

1: Initialize  $\overleftarrow{m}_{Y_i} = 0$ ,  $\overleftarrow{V}_{Y_i} = 1$  and  $\gamma_i = 10^{-3}$  for  $i = 1, \dots, n$ 
2: while not converged do
3:   Standard forward Kalman filtering:
4:   Set  $\overrightarrow{m}_{X_1}$  and  $\overrightarrow{V}_{X_1}$  according to  $\overrightarrow{\mu}_{X_1}$ 
5:   for  $i = 1$  to  $n$  do
6:      $G_i = (\overleftarrow{V}_{Y_i} + C_{i,\cdot} \overrightarrow{V}_{X_i} C_{i,\cdot}^\top)^{-1}$ 
7:      $\overrightarrow{V}_{X_{i+1}} = \overrightarrow{V}_{X_i} - \overrightarrow{V}_{X_i} C_{i,\cdot}^\top G_i C_{i,\cdot} \overrightarrow{V}_{X_i}$ 
8:      $\overrightarrow{m}_{X_{i+1}} = \overrightarrow{m}_{X_i} + \overrightarrow{V}_{X_i} C_{i,\cdot}^\top G_i (\overleftarrow{m}_{Y_i} - C_{i,\cdot} \overrightarrow{m}_{X_i})$ 
9:   end for
10:  Backward dual deciding:
11:   $\hat{x}_{n+1} = 0$ 
12:  for  $i = n$  to  $1$  do
13:     $\overrightarrow{V}_{Y_i} = C_{i,\cdot} \overrightarrow{V}_{X_i} C_{i,\cdot}^\top$ 
14:     $\overrightarrow{m}_{Y_i} = C_{i,\cdot} (\overrightarrow{m}_{X_i} - \overrightarrow{V}_{X_i} \hat{x}_{i+1})$ 
15:    Decide  $\hat{y}_i$  and update  $\gamma_i$  with Table 2.2
16:    Update  $\overleftarrow{m}_{Y_i}$  and  $\overleftarrow{V}_{Y_i}$  with Table 2.2
17:     $\hat{x}_i = \hat{x}_{i+1} + C_{i,\cdot}^\top \hat{y}_i$ 
18:  end for
19:   $\hat{x}_1 = \overrightarrow{m}_{X_1} - \overrightarrow{V}_{X_1} \hat{x}_1$ 
20:  Update  $\overrightarrow{\mu}_{X_1}$  according to the modeled prior.
21: end while

```

---

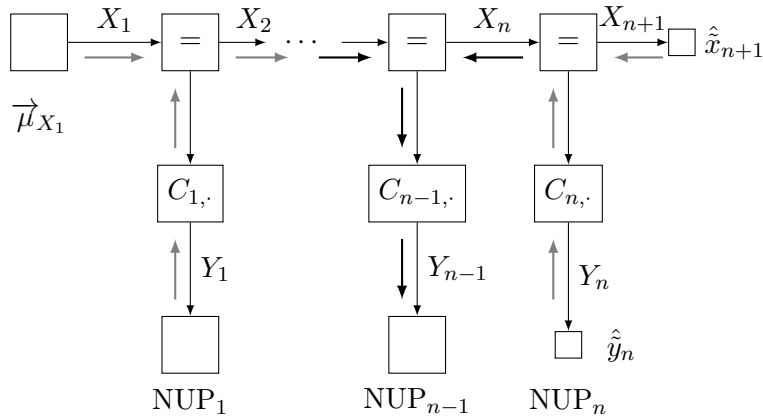


Figure 2.5: IFFBDD in backward dual deciding phase.

### 2.2.4 On The Internal NUP Parameter $\gamma$

The authors in [1] proposed that we can start Algorithm 2 by initializing all  $\gamma$  with  $\gamma > 0$ . As mentioned in [1, Appendix A], we cannot work directly with the dual cost function  $\kappa^*$  but need a proxy  $\tilde{\kappa}$  with an internal parameter  $\gamma$ . We briefly mentioned in Section 2.2.2 that we need to adjust  $\gamma$  such that the two cost functions coincide at the relevant part. Let us analyze this further using the example of the  $L_1$  cost function  $\kappa$ . We list its dual  $\kappa^*$  and proxy  $\tilde{\kappa}$  in Table 2.3. Inspecting  $\gamma$  in the proxy cost, we can interpret  $\gamma$  as the slope of the proxy cost function. Examining the proxy function  $\tilde{\kappa}$ , it aligns with the dual  $\kappa^*$  in the middle row. Therefore, we must ensure that  $\tilde{y} \in [-\beta, \beta]$  for the NUP to represent the dual cost. In [4, Section 3.4], we have the option to adjust our slope  $\gamma$  when deciding, such that our decided value falls within the desired region  $[-\beta, \beta]$ . This is where the updates for the  $\gamma$  in [1, Appendix A] and, in Table 2.1 and Table 2.2 originate. So we can conclude that, if necessary, increasing the dual slope parameter  $\gamma$  ensures that the dual decision  $\hat{y}$  lands on the border of the required region of  $\kappa^*$  while using a proxy cost function  $\tilde{\kappa}$ . Now we can examine two interesting special cases for the initialization of  $\gamma$  and give interpretations to them.

**Initialize  $\gamma = 0$ :** When we look at the update rules for the  $\gamma$  in Table 2.2 and particularly observe  $\overleftarrow{W}_Y$  in the primal domain, we can interpret  $\gamma$  as a notion of wiggle room. How much wiggle room does this particular observation allow? A small  $\gamma$  leads to high precision  $\overleftarrow{W}_Y$ , which means little wiggle room. The fact that we increase  $\gamma$  if and only if the incoming Gaussian message  $\overrightarrow{\mu}_Y$  already satisfies the constraint by some margin supports this loose hypothesis. Using this intuition, we can argue that it makes sense to start our algorithm with no wiggle room at all, i.e.,  $\gamma = 0$ . Initializing  $\gamma$  with zero means that at first, our proxy costs  $\tilde{\kappa}$  represent only the middle row of the dual costs  $\kappa^*$ . This is fine as long as the incoming Gaussian message already leads to a decision within the desired region; otherwise, we increase  $\gamma$  as necessary. This "free" decision in the dual domain, however, leads to a restricted variable in the primal domain. Which, in our case, might be desirable. For further information on constrained problems in the primal domain and unconstrained problems in the dual domain, consider [5, Chapter 5.2, The Lagrange dual problem]. We can implement this idea of initializing  $\gamma$  with zero as long as

$$C_{i,\cdot} \overrightarrow{V}_{X_i} C_{i,\cdot}^\top \neq 0. \quad (2.43)$$

If we had  $C_{i,\cdot} \overrightarrow{V}_{X_i} C_{i,\cdot}^\top = 0$ , both quantities in Line 6 in the denominator of  $G_i$  in Algorithm 2 would be zero, and then we run into analytical and numerical issues. However, all the numerical experiments for constrained optimization did not encounter such a problem, and starting with  $\gamma = 0$  is effective.

| Constraint            | $Y = c$                     |
|-----------------------|-----------------------------|
| $\hat{y}$             | $(\vec{m}_Y - c)/\vec{V}_Y$ |
| $\overleftarrow{m}_Y$ | $c$                         |
| $\overleftarrow{V}_Y$ | $0$                         |

Table 2.4: Update rules for equality constraint with  $\gamma = 0$ .

| Equality Constraint | Table 2.2              | Table 2.4             |
|---------------------|------------------------|-----------------------|
| Mean Iterations     | 296.35                 | 4                     |
| Minimum Iterations  | 46                     | 4                     |
| Maximum Iterations  | 1443                   | 4                     |
| Mean Violation      | $9.2 \cdot 10^{-14}$   | $9.0 \cdot 10^{-14}$  |
| Maximum Violation   | $3.943 \cdot 10^{-13}$ | $3.26 \cdot 10^{-13}$ |

Table 2.5: Comparison of equality constraint methods for repeated experiment with 100 equality constraints and  $x \in \mathbb{R}^{100}$ .

If we have an equality constraint, this initialization also makes much informal sense, because an equality constraint requires the algorithm to land exactly at that point. It is thus a constrained problem with no allowed wiggle room at that particular observation. We can even adjust the update rule for an equality constraint  $Y = c$  as depicted in Table 2.4. Especially when the objective in Equation (2.1) has many equality constraints, using the update rule as in Table 2.4 improves the convergence speed. We see a drastic example in Table 2.5, where we improved the convergence speed with the new update rules without compromising the quality of the result. It remains to be examined if we could also initialize  $\gamma_i$  with a minimal value, close to numerical precision, and also achieve similar speed improvements.

Line 6 and Line 7 in Algorithm 2 both rely on the Matrix Inversion Lemma [3, Theorem C.8]. It is interesting that starting with  $\gamma = 0$  works, despite violating the conditions of the Matrix Inversion Lemma, because it requires  $\vec{V}_{Y_i}$  to be invertible. Upon closer examination, we observe that we want to calculate

$$\vec{V}_{X_{i+1}} = (\vec{W}_{X_i} + \vec{W}_{Y_i} C_{i,\cdot}^\top C_{i,\cdot})^{-1}. \quad (2.44)$$

This is exactly the expression given in [6, Sherman-Morrison formula], which has the milder conditions that  $\vec{W}_{X_i}$  must be invertible and

$$1 + \vec{W}_{Y_i} C_{i,\cdot} \vec{V}_{X_i} C_{i,\cdot}^\top \neq 0. \quad (2.45)$$

| Cost                                                                                                 | Case                                                 | $\hat{y} = -\overleftarrow{\xi}_Y$ |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------|
| $\beta y - a $                                                                                       | $(\vec{m}_Y - a)/\vec{V}_Y < -\beta$                 | $-\beta$                           |
|                                                                                                      | $-\beta \leq (\vec{m}_Y - a)/\vec{V}_Y \leq \beta$   | $(\vec{m}_Y - a)/\vec{V}_Y$        |
|                                                                                                      | $(\vec{m}_Y - a)/\vec{V}_Y > \beta$                  | $\beta$                            |
| $\begin{cases} \beta(a - y), & y < a \\ 0, & y \geq a \end{cases}$                                   | $(\vec{m}_Y - a)/\vec{V}_Y < -\beta$                 | $-\beta$                           |
|                                                                                                      | $-\beta \leq (\vec{m}_Y - a)/\vec{V}_Y \leq 0$       | $(\vec{m}_Y - a)/\vec{V}_Y$        |
|                                                                                                      | $(\vec{m}_Y - a)/\vec{V}_Y > 0$                      | $0$                                |
| $\begin{cases} 0, & y \leq b \\ \beta(y - b), & y > b \end{cases}$                                   | $(\vec{m}_Y - b)/\vec{V}_Y < 0$                      | $0$                                |
|                                                                                                      | $0 \leq (\vec{m}_Y - b)/\vec{V}_Y \leq \beta$        | $(\vec{m}_Y - b)/\vec{V}_Y$        |
|                                                                                                      | $(\vec{m}_Y - b)/\vec{V}_Y > \beta$                  | $\beta$                            |
| $\begin{cases} 2\beta(a - y), & y < a \\ 0, & a \leq y \leq b \\ 2\beta(y - b), & y > b \end{cases}$ | $(\vec{m}_Y - a)/\vec{V}_Y < -2\beta$                | $-2\beta$                          |
|                                                                                                      | $-2\beta \leq (\vec{m}_Y - a)/\vec{V}_Y < 0$         | $(\vec{m}_Y - a)/\vec{V}_Y$        |
|                                                                                                      | $a/\vec{V}_Y \leq \vec{m}_Y/\vec{V}_Y < b/\vec{V}_Y$ | $0$                                |
|                                                                                                      | $0 \leq (\vec{m}_Y - b)/\vec{V}_Y < 2\beta$          | $(\vec{m}_Y - b)/\vec{V}_Y$        |
|                                                                                                      | $2\beta \leq (\vec{m}_Y - b)/\vec{V}_Y$              | $2\beta$                           |

Table 2.6: Update rules for dual costs with  $\gamma \rightarrow \infty$ .

**Initialize  $\gamma \rightarrow \infty$ :** We can also argue in the other direction. Since we sometimes need to increase  $\gamma$  such that the dual cost  $\kappa^*$  and the proxy cost  $\tilde{\kappa}$  coincide at the relevant sections, why should we not let  $\gamma \rightarrow \infty$  in the initialization? Then, the two costs always coincide, regardless of the incoming Gaussian message  $\vec{\mu}_Y$ .

We can apply this idea, allowing us to simplify Algorithm 2. Inspecting Table 2.1 and Table 2.2, we notice that we obtain  $\overleftarrow{W}_Y = 0$  and  $\hat{y} = -\overleftarrow{\xi}_Y$  for  $\gamma \rightarrow \infty$ . The result that  $\overleftarrow{W}_Y = 0$  implies that the NUPs do not add any curvature to the Kalman filtering step. We present the simplified updates in Table 2.6 and Table 2.7. Now looking at Algorithm 2, we notice that for  $\overleftarrow{W}_{Y_i} \rightarrow 0$  we get  $G_i \rightarrow 0$ . Thus,  $\vec{V}_{X_i} = \vec{V}_{X_1}$  for all  $i$ , since the scalar  $G_i$  in the rank-one update on Line 7 is zero.

| Constraint                | Case               | $\hat{y} = -\overleftarrow{\xi}_Y$ |
|---------------------------|--------------------|------------------------------------|
| $Y \geq a$                | $\vec{m}_Y \geq a$ | 0                                  |
|                           | $\vec{m}_Y < a$    | $(\vec{m}_Y - a)/\vec{V}_Y$        |
| $Y \leq b$                | $\vec{m}_Y \leq b$ | 0                                  |
|                           | $\vec{m}_Y > b$    | $(\vec{m}_Y - b)/\vec{V}_Y$        |
| $a \leq \vec{m}_Y \leq b$ |                    | 0                                  |
| $a \leq Y \leq b$         | $\vec{m}_Y < a$    | $(\vec{m}_Y - a)/\vec{V}_Y$        |
|                           | $\vec{m}_Y > b$    | $(\vec{m}_Y - b)/\vec{V}_Y$        |
| $Y = c$                   | no distinction     | $(\vec{m}_Y - c)/\vec{V}_Y$        |

Table 2.7: Update rules for constraints with  $\gamma \rightarrow \infty$ .

Nevertheless, the limit for the mean update  $\vec{m}_{X_{i+1}}$  in Line 8 exists, and is given by:

$$\vec{m}_{X_{i+1}} = \lim_{\overleftarrow{W}_{Y_i} \rightarrow 0} \vec{m}_{X_i} + G_i \vec{V}_{X_1} C_{i,\cdot}^\top (\overleftarrow{m}_{Y_i} - C_{i,\cdot} \vec{m}_{X_i}) \quad (2.46)$$

$$= \lim_{\overleftarrow{W}_{Y_i} \rightarrow 0} \vec{m}_{X_i} + \frac{\overleftarrow{W}_{Y_i}}{1 + \overleftarrow{W}_{Y_i} C_{i,\cdot} \vec{V}_{X_1} C_{i,\cdot}^\top} \vec{V}_{X_1} C_{i,\cdot}^\top (\overleftarrow{m}_{Y_i} - C_{i,\cdot} \vec{m}_{X_i}) \quad (2.47)$$

$$= \lim_{\overleftarrow{W}_{Y_i} \rightarrow 0} \vec{m}_{X_i} + \vec{V}_{X_1} C_{i,\cdot}^\top (\overleftarrow{W}_{Y_i} \overleftarrow{m}_{Y_i} - \overleftarrow{W}_{Y_i} C_{i,\cdot} \vec{m}_{X_i}) \quad (2.48)$$

$$= \lim_{\overleftarrow{W}_{Y_i} \rightarrow 0} \vec{m}_{X_i} + \vec{V}_{X_1} C_{i,\cdot}^\top \overleftarrow{\xi}_{Y_i} \quad (2.49)$$

$$= \vec{m}_{X_i} - \vec{V}_{X_1} C_{i,\cdot}^\top \hat{y}_i. \quad (2.50)$$

With the expression in Equation (2.50), we can rewrite a simplified IFF-BDD as given in Algorithm 3.

Notice in Algorithm 3 that if  $\vec{\mu}_{X_1}(x_1) = \mathcal{N}(x_1; 0, I)$ , i.e., we want to minimize the Euclidean norm, we can disregard the covariance  $\vec{V}_{X_1}$ , which makes the algorithm extremely cheap. Furthermore, this version of the algorithm coincides with the *dual coordinate descent* algorithm, which is a state-of-the-art algorithm [7]. Still, if  $\vec{V}_{X_1}$  is diagonal, which corresponds to the  $L_1$  norm, we can use a vector to represent  $\vec{V}_{X_1}$ . We find, that in Line 6, Line 11 and Line 12, the matrix  $\vec{V}_{X_1}$  is always diagonal. Thus, this matrix-vector multiplication corresponds to a vector-vector multiplication, where we perform a component-wise multiplication, i.e., the Hadamard product. This is computationally much more attractive.



**Algorithm 3** IFFBDD with infinite  $\gamma$ 


---

```

1: Initialize  $\hat{y}_i = 0$  for all  $i = 1 : n$ 
2: while not converged do
3:   Standard forward Kalman filtering:
4:   Set  $\vec{m}_{X_1}$  and  $\vec{V}_{X_1}$  according to  $\vec{\mu}_{X_1}$ 
5:   for  $i = 1$  to  $n$  do
6:      $\vec{m}_{X_{i+1}} = \vec{m}_{X_i} - \vec{V}_{X_1} C_{i,\cdot}^\top \hat{y}_i$ 
7:   end for
8:   Backward dual deciding:
9:    $\hat{x}_{n+1} = 0$ 
10:  for  $i = n$  to  $1$  do
11:     $\vec{V}_{Y_i} = C_{i,\cdot} \vec{V}_{X_1} C_{i,\cdot}^\top$ 
12:     $\vec{m}_{Y_i} = C_{i,\cdot} (\vec{m}_{X_i} - \vec{V}_{X_1} \hat{x}_{i+1})$ 
13:    Decide  $\hat{y}_i$  with Table 2.6 and Table 2.7
14:     $\hat{x}_i = \hat{x}_{i+1} + C_{i,\cdot}^\top \hat{y}_i$ 
15:  end for
16:   $\hat{x}_1 = \vec{m}_{X_1} - \vec{V}_{X_1} \hat{x}_1$ 
17:  Update  $\vec{\mu}_{X_1}$  according to the modeled prior.
18: end while

```

---

With Algorithm 3, we have presented a first-order method. A first-order method approximates the solution by fitting a tangent plane, whereas a second-order method — such as Algorithm 2 — incorporates curvature by fitting a parabola.

## 2.3 Comparing Algorithms

### 2.3.1 On Minimizing $L_1$ Cost

In this section, we present two ways to model the  $L_1$  cost in Equation (2.1), i.e., we aim minimize

$$\|x\|_1 = \sum_{i=1}^k |x_i|. \quad (2.51)$$

For completeness, we will state two other priors at the end of this section, but the primary focus lies on  $|x|_1$ . Additionally, note that the following discussion assumes we are in the setting of  $\gamma \rightarrow \infty$ , as in Algorithm 3. It remains to be explored how the two techniques compare in Algorithm 2.

We can take the first technique from [3, Theorem 6.1]. By using the estimate  $\hat{x}_1$  we can adjust the prior  $\vec{\mu}_{X_1}$  in Algorithm 3 by updating each diagonal entry of the forward covariance matrix with

$$\sigma_i^2 = |(\hat{x}_1)_i|, \quad (2.52)$$

where

$$\hat{x}_1 = \vec{m}_{X_1} - \vec{V}_{X_1} \hat{\hat{x}}_1 \quad (2.53)$$

$$= -\vec{V}_{X_1} \hat{\hat{x}}_1. \quad (2.54)$$

The forward mean  $\vec{m}_{X_1}$  is zero and remains unchanged for all iterations. Assuming the candidate solution  $\hat{x}_1$  from the previous iteration was  $x_0 \in \mathbb{R}^k$ , we get the next forward message  $\vec{\mu}_{X_1}$  as

$$\vec{V}_{X_1} = \text{diag}(|\hat{x}_1|) \quad (2.55)$$

$$= \text{diag}(|\text{diag}(x_0) \cdot \hat{\hat{x}}_1|), \quad (2.56)$$

$$\vec{m}_{X_1} = 0, \quad (2.57)$$

where  $|\cdot|$  should be understood elementwise. The candidate solution is given by

$$\hat{x}_1 = -\text{diag}(x_0) \cdot \hat{\hat{x}}_1. \quad (2.58)$$

This method thus minimizes the  $L_1$  cost of  $x$  by alternating maximization, consider further [3, Section 6.3].

The second technique utilizes the dual NUP boxes; we can prepend (or append) a chain of  $L_1$  costs to the chain of constraints, where  $C_i = e_i$  and  $e_i$  is the  $i$ -th standard basis vector. We will use Figure 2.6 as a zoomed-in representation of the prior  $\vec{\mu}_{X_1}$ . The box  $\vec{\mu}_{X_0}$  is still an NUP box, but we need to be careful about the cost this NUP model incurs. Since the chain of  $L_1$  costs introduces our objective, this cost must be *neutral*. We can get a *neutral* cost by modeling a uniform prior. Modeling a uniform prior with Gaussian messages is not directly possible; however, we can approximate the uniform prior by adjusting the mean  $\vec{m}_{X_0}$  of  $\vec{\mu}_{X_0}$  to

$$\vec{m}_{X_0} = \hat{x}_0 \quad (2.59)$$

and leaving  $\vec{V}_{X_0} = I$ .

This approach always results in an identity-forward covariance matrix from the prior. For Algorithm 2, this does not offer many benefits. However, for the first-order method, i.e., Algorithm 3, it provides storage and computational benefits. Consider further Section 2.2.4 and the analysis in Section 2.3.2.

Since the linear mappings are the standard basis vectors  $e_j$ , each  $L_1$  cost acts independently on one of the elements of  $X_0$  (or  $X_1$ ). In the setting of Algorithm 3, we thus obtain that the Gaussian message  $\vec{\mu}_{Z_j}$  into the  $j$ -th  $L_1$  as

$$\vec{V}_{Z_j} = 1, \quad (2.60)$$

$$\vec{m}_{Z_j} = e_j \left( \vec{m}_{X_0} - \vec{V}_{X_0} \hat{\hat{x}}_1 - \vec{V}_{X_0} \sum_{i=j+1}^k e_i^\top \hat{\hat{z}}_i \right) \quad (2.61)$$

$$= (\vec{m}_{X_0})_j - (\hat{\hat{x}}_1)_j. \quad (2.62)$$

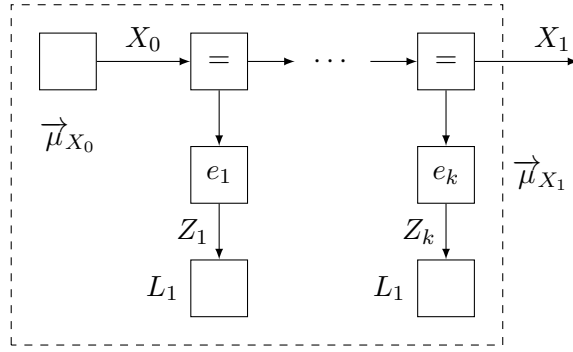


Figure 2.6: Factor graph of  $L_1$  objective prior box with dual  $L_1$  costs for Algorithm 3.

Inspecting  $L_1$  costs in Table 2.6 with  $c = 0$  and  $\beta = 1$ , we note that  $\hat{z}_j$  gets decided as

$$\hat{z}_j = \text{proj}_{[-1,1]}(\vec{m}_{Z_j}), \quad (2.63)$$

where

$$\text{proj}_{[a,b]}(z) = \begin{cases} a, & z < a, \\ z, & a \leq z \leq b, \\ b, & z > b. \end{cases} \quad (2.64)$$

Assuming now that we have  $\vec{m}_{X_0} = x_0$  from the previous iteration, we get that

$$\hat{x}_0 = \hat{x}_1 + \sum_{i=1}^k e_i^\top \hat{z}_i \quad (2.65)$$

$$= \hat{x}_1 + \text{proj}_{[-1,1]}(\vec{m}_{X_0} - \hat{x}_1), \quad (2.66)$$

where the  $\text{proj}_{[-1,1]}(\cdot)$  should be understood element-wise. Thus, we get the candidate solution as

$$\hat{x}_0 = \vec{m}_{X_0} - \vec{V}_{X_0} \hat{x}_1 \quad (2.67)$$

$$= x_0 - \hat{x}_1 - \text{proj}_{[-1,1]}(x_0 - \hat{x}_1). \quad (2.68)$$

After that, we can continue with the next forward filtering step and write  $\vec{\mu}_{X_1}$  as

$$\vec{V}_{X_1} = I, \quad (2.69)$$

$$\vec{m}_{X_1} = \vec{m}_{X_0} - \vec{V}_{X_0} \sum_{i=1}^k e_i^\top \hat{z}_i \quad (2.70)$$

$$= \hat{x}_0 - \text{proj}_{[-1,1]}(x_0 - \hat{x}_1) \quad (2.71)$$

$$= x_0 - \hat{x}_1 - 2\text{proj}_{[-1,1]}(x_0 - \hat{x}_1). \quad (2.72)$$

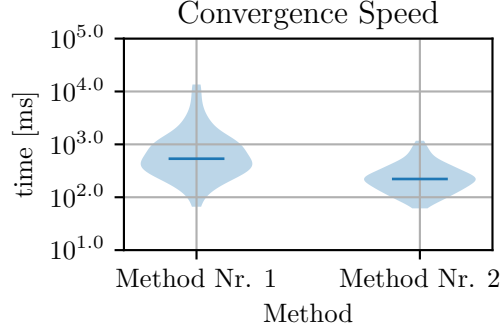


Figure 2.7: Convergence speed of different  $L_1$  prior methods. The distribution of convergence time is depicted in light blue. The blue bar indicates the median.

To summarize, we obtain the next candidate solution in Equation (2.68) and the forward message  $\vec{\mu}_{X_1}$  as given in Equation (2.69) and Equation (2.72). Remember that  $x_0$  is the candidate solution of the previous iteration. This method minimizes the  $L_1$  cost of  $x$  by dual decisions.

A final note: For the first-order method, we need the "uniform" prior NUP  $\vec{\mu}_{X_0}$ , because no other constraint or cost introduces variance. But for Algorithm 3 to work, we need at least one box, which has a Gaussian message with a well-defined variance.

Comparing the techniques, especially the candidate  $\hat{x}_1$  in Equation (2.58) to  $\hat{x}_0$  in Equation (2.68), we can conclude that the two methods operate differently and produce distinct Gaussian messages  $\vec{\mu}_{X_1}$  that enter the chain of constraints.

For an experiment repeated 100 times with  $k = 100$  and 70 constraints, we depict the convergence speed in milliseconds for the first-order algorithms in Figure 2.7. The first method involves changing the diagonal covariance matrix, and the second method involves appending dual  $L_1$  costs. Note that we cannot compare the total number of iterations between the two methods, because finishing an iteration in the second technique takes more computational work due to the appended  $L_1$  costs. What is not visible here is that the second technique converges to candidates  $\hat{x}_0$  that violate the constraints more severely than the first technique. However, for our experiments, the violations were  $\approx 10^{-6}$ , which is in an acceptable range. Additionally, we could speed up the second technique by implementing the update rules directly, as derived above. We did not implement it in this experiment for ease of implementation. Simply appending a chain of  $L_1$  costs is very straightforward. The convergence speed for the implemented analytical rule remains to be tested.

Finally, we state two other priors, with which we could model  $L_2$  and  $L_p$ :

- The  $L_2$  cost, i.e., minimizing

$$\|x\|_2^2 = \sum_{i=1}^k x_i^2. \quad (2.73)$$

For this objective, we set  $\vec{\mu}_{X_1} = \mathcal{N}(0, I)$  and, through the iterations, we do not need to update this box in either Algorithm 2 or Algorithm 3.

- [2, Theorem 6.1] also offers us a method to model  $L_p$  costs for  $p > 2$ , i.e.,

$$\|x\|_p^p = \sum_{i=1}^k |x|^p. \quad (2.74)$$

However, as also stated in [3, Section 6.2.3], this turned out to be numerically unstable. We can see this by the update rule,

$$\sigma_i^2 = \frac{|(x)_i|^{2-p}}{p}, \quad (2.75)$$

for  $p > 2$ . If some  $x$  is close to 0, we have an exploding variance, which cannot be represented with enough precision by a floating-point number. Thus, we discarded the implementation of this prior option.

### 2.3.2 Iteration Scaling

We can make a rough analysis of the computational complexity of the three algorithms presented above. For this, we need a measure for the linear algebra operations performed in the different pseudo-codes. In the following, we consider  $a$  and  $b$  to be scalars,  $x$  and  $y$  to be  $n$ -dimensional vectors, and  $C$  to be an  $n \times n$  matrix. We consider both the addition and multiplication of two scalars to be a single floating-point operation (FLOP). Now we can count the FLOPs of different linear algebra operations used in our pseudo-codes, to give an outline of the computational complexity in terms of the dimensions.

- $y + ax$ : We have  $n$  operations to multiply each element of  $x$  by  $a$  and then  $n$  operations to add the elements from the two vectors, thus  $y + ax$  takes  $\approx 2n$  flops.
- $x^\top y$ : We perform  $n$  elementwise multiplications and then perform  $n - 1$  summations of the results, thus  $x^\top y$  needs  $\approx 2n$  flops.
- $Cx$ : Per row  $C_{i,\cdot}x$  we perform  $\approx 2n$  flops. Since there are  $n$  rows, we get that  $Cx$  requires  $\approx 2n^2$  flops.
- $C + axy^\top$ : We perform  $n$  multiplications to calculate  $ax$ , then for each element  $y_i$  we multiply  $ax$ , thus we get  $axy^\top \approx n + n^2$  flops, then adding the resulting matrix to  $C$  takes again  $n^2$  flops, thus  $C + axy^\top$  takes  $\approx 2n^2 + n$  flops.

- $aCx + by$ :  $by$  and  $ax$  both take  $n$  operations, then  $Cx$  takes  $2n^2$  flops, and adding the two vectors again takes  $n$  flops, thus  $aCx + by$  needs  $\approx 2n^2 + 3n$  flops.

Now we can count how many flops a single iteration costs in the different algorithms. Please note that we only consider operations that scale with the increasing dimension of  $x$ ; for example, we disregard the cost of calculating the reciprocal of a scalar. We presented these matrix-vector operations because, in the implementation, we used BLAS, which provides these operations. For more information about BLAS, consider Section 4.4.

We now assume that we are in the setting of Equation (2.1) and aim to minimize  $\|x\|_2^2$ . We have  $C$ , which is an  $n \times k$  matrix; thus, we have  $n$  constraints, and  $x$  will be a  $k$ -dimensional vector.

- Algorithm 1 (IRLGE):  
To update the upward messages in each constraint, we must compute a single dot product, which costs  $\approx 2k$  flops. A single forward Kalman filtering step consists of the following operations:  $1 \times (aCx + by)$ ,  $2 \cdot (x^\top y)$ ,  $1 \cdot (y + ax)$ ,  $1 \cdot (C + axy^\top)$ . Thus, one iteration of the IRLGE Algorithm takes  $\approx n \cdot (4k^2 + \mathcal{O}(k))$  flops. Note that such an iteration is not a complete inner loop but only one iteration of it. This algorithm performs iterations until convergence in the inner loop and then updates the slopes  $\beta_i$ , which constitutes one step of the outer loop.
- Algorithm 2 (IFFBDD):  
As in the IRLGE Algorithm, we make a forward Kalman filtering pass, and the computational complexity is the same as above. For the backward dual deciding, we can reuse many computed quantities from the forward pass. The only two additional operations we need to perform are  $1 \cdot (y + ax)$ , and  $1 \cdot (x^\top y)$ ; thus, one iteration of the IFFBDD Algorithm takes  $\approx n \cdot (4k^2 + \mathcal{O}(k))$  flops.
- Algorithm 3 (IFFBDD with infinite  $\gamma$ ):  
For each constraint, we can precompute  $\vec{V}_{Y_i}$  because  $\vec{V}_{X_1} = I$ ; thus, we do not need to take this into account. Then the forward pass does a single vector addition  $1 \times (y + ax)$  and the backward pass has to perform the operations  $2 \cdot (y + ax)$ ,  $1 \cdot (x^\top y)$ , thus one iteration of the IFFBDD with infinite  $\gamma$  algorithm takes  $\approx n \cdot (8k + \mathcal{O}(1))$  flops.

We can do a similar analysis for the memory used. We will count the number of double-precision floats required in memory. Each such float is an 8-byte long data structure. We will only employ a crude approximation here; for instance, we do not account for small temporary values. However, we still ensure that temporary array allocations do not occur, as discussed in Section 4.1. Furthermore, we will only consider memory that scales with the dimension  $k$  of  $x$  or the number of constraints  $n$ . We will not consider the linear mapping  $C_{i,\cdot} \in \mathbb{R}^k$  and the border  $b_i \in \mathbb{R}$ , as these are not bound to a specific algorithm.

- Algorithm 1 (IRLGE):

The algorithm needs space for the forward mean vector  $\vec{m}_{X_i} \in \mathbb{R}^k$  and forward covariance matrix  $\vec{V}_{X_i} \in \mathbb{R}^{k \times k}$ . In each constraint, we have to store two scalar messages,  $\overleftarrow{m}_{Y_i} \in \mathbb{R}$  and  $\overleftarrow{V}_{Y_i} \in \mathbb{R}$ , and the slope  $\beta_i \in \mathbb{R}$ . Thus, this algorithm needs allocated memory for  $\approx n \cdot 3 + k^2 + k$  floats.

- Algorithm 2 (IFFBDD):

The algorithm needs space for the forward mean vector  $\vec{m}_{X_i} \in \mathbb{R}^k$  and forward covariance matrix  $\vec{V}_{X_i} \in \mathbb{R}^{k \times k}$ . For the backward pass  $\hat{x}_i$ , we can reuse the mean vector. In each constraint, we have to store two scalar messages for the forward pass  $\overleftarrow{m}_{Y_i} \in \mathbb{R}$  and  $\overleftarrow{V}_{Y_i} \in \mathbb{R}$ . We reuse this memory for the backward pass, where we stored  $C_{i,\cdot} \vec{m}_{X_i} \in \mathbb{R}$  and  $C_{i,\cdot} \vec{V}_{X_i} C_{i,\cdot}^\top \in \mathbb{R}$ . Additionally, we need to store  $\gamma_i \in \mathbb{R}$  and  $\vec{V}_{X_i} C_{i,\cdot}^\top \in \mathbb{R}^k$ . Thus, this algorithm needs allocated memory for  $\approx n \cdot (3 + k) + k^2 + k$  floats.

- Algorithm 3 (IFFBDD with infinite  $\gamma$ ):

The algorithm needs space for the forward mean vector  $\vec{m}_{X_i} \in \mathbb{R}^k$  and the backward message  $\hat{x}_i$ . In each constraint, we have to store a scalar message for the forward pass  $\hat{y} \in \mathbb{R}$ . We reuse this memory for the backward pass, where we store  $C_{i,\cdot} \vec{m}_{X_i}$ . Depending on the objective we solve and the method we use to solve it, we require a different amount of space for the covariance method.

**Minimize  $L_2$ :** Here, we do not need a covariance matrix, but we need to store a constant  $\vec{V}_{Y_i} \in \mathbb{R}$ . Thus, this algorithm needs allocated memory for  $\approx n \cdot 2 + k$  floats.

**Minimize  $L_1$  with diagonal covariance matrix:** Here we need a diagonal covariance matrix  $\vec{V}_{X_1} \in \mathbb{R}^k$ . For each constraint we also need to store  $C_{i,\cdot} \vec{V}_{X_1} \in \mathbb{R}^k$  and the variance  $\vec{V}_{Y_i} \in \mathbb{R}$ . Thus, this algorithm requires allocated memory for  $\approx n \cdot (2 + k) + 2k$  floats.

**Minimize  $L_1$  with dual  $L_1$  costs:** As in the  $L_2$  case, we do not need a covariance matrix here, and  $\vec{V}_{Y_i} \in \mathbb{R}$  is constant. Thus, this algorithm requires allocated memory for  $\approx n \cdot 2 + k$  floats.

Note, we only achieve this slim memory footprint if we use the analytically derived update rules for the prior. If we append a chain of dual  $L_1$  costs with  $e_i$  as the linear mapping, we require more memory.

We notice that Algorithm 3 benefits greatly if the objective in Equation (2.1) is  $\|x\|_2^2$  or if we use the second method to minimize  $\|x\|_1$ . Especially, the required memory per constraint increases if we have a covariance matrix that is not the identity.

### 2.3.3 Convergence Speed

We now do some comparisons between the different presented algorithms. Unless otherwise stated, the objective at hand is to minimize the Euclidean norm  $\|x\|_2^2$ . Some results also show a comparison with the commercial solver Gurobi [8]. Gurobi uses a simplex or barrier algorithm to solve our objective given in Equation (2.1). For further information, consolidate the provided documentation of Gurobi [8, Section Quadratic Objectives]. Additionally, it performs preprocessing steps on a given problem. This further reduces the time required to solve a problem. We do not perform such preprocessing steps; our focus lies on the different algorithms. Note that even though we make speed comparisons here, the results may differ on other machines because there are external factors that we cannot test, such as cache management. Thus, we attempted to make more meaningful comparisons, such as the number of iterations. To check for convergence in our algorithms, we verify if the absolute difference in the norms of our candidate solution between the iterations is below a threshold *thresh*. For Algorithm 1 this is  $\hat{x}$ , for Algorithm 2 and Algorithm 3 it is  $\hat{x}_1$ . So we check if

$$| \|(\hat{x}_1)_{\text{new}}\|_2^2 - \|(\hat{x}_1)_{\text{old}}\|_2^2 | \leq \text{thresh}. \quad (2.76)$$

For Algorithm 2, we also verify this difference for internal parameters, i.e.,  $\sum_i \gamma_i^2$ .

In Figure 2.8, we depict the three algorithms using Gurobi as the reference solver. We can observe the previously described problem that Algorithm 1 is slow in convergence. Both IFFBDD algorithms work very well in this particular setup. However, if we increase the scale of  $n$  or  $k$ , the Gurobi solver will be faster than the IFFBDD methods. Part of the improved performance of the Gurobi solver is due to the preprocessing it performs. But notably, Algorithm 3 performs comparably to the Gurobi solver even for larger-scale problems. As calculated in Section 2.3.2, this is due to the good computational scaling. We chose this setup to compare the IFFBDD algorithms with the IRLGE algorithm, which would take much longer for larger dimensions. Additionally, note that the squared problem, i.e.,  $C \in \mathbb{R}^{k \times k}$ , is the most difficult problem to solve.

Figure 2.9 illustrates the evolution of the different algorithms until convergence. This plot can be insightful to see the behavior of the different algorithms. Note that Algorithm 1 (IRLGE) displays the outer loop; specifically, each iteration shows the converged inner loop after we have updated the slope parameters  $\beta_i$ . We also notice that the different algorithms all take a long time until convergence is declared. This suggests that if we do not require such exact solutions, we may be able to stop much earlier.

Figure 2.10 depicts the evolution of the costs and violations of Algorithm 1 for an inner loop with fixed slopes  $\beta_i$ . We can observe that the cost  $\|x\|_2^2$  does not decrease in every iteration, even though the problem is convex. That is because we modeled the constraints as costs. Consider further Figure 2.1.



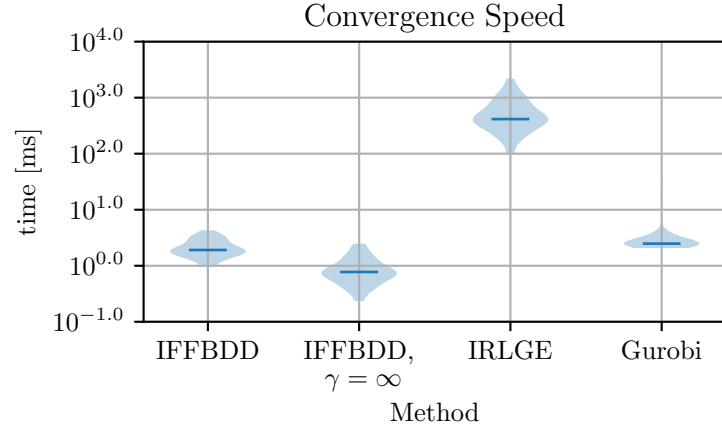


Figure 2.8: Convergence speed for repeated experiment with  $k = 50$  and 50 constraints. The distribution of convergence time is depicted in light blue. The blue bar indicates the median.

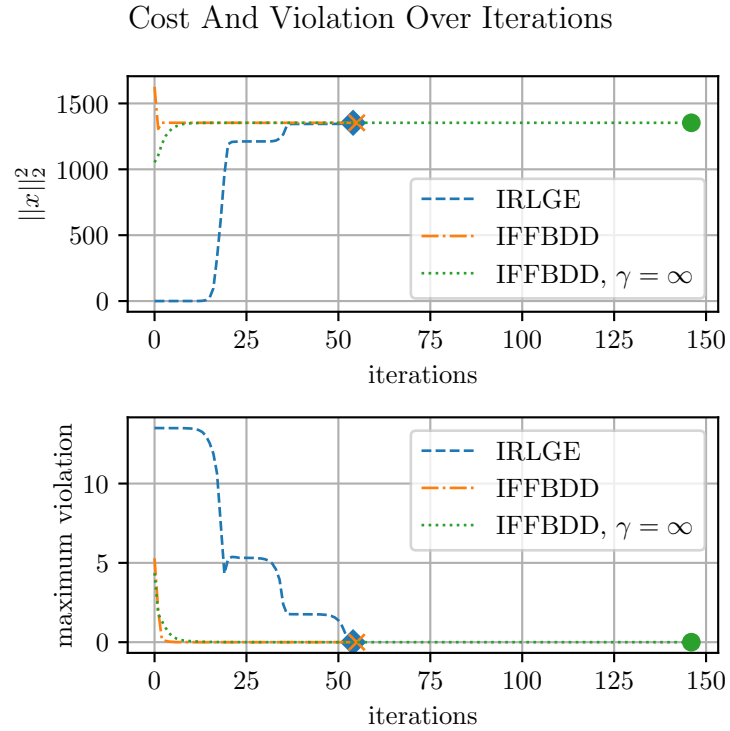


Figure 2.9: Violation and cost evolution over the iterations of the algorithms. The experiment was  $k = 100$  and 100 constraints.

Cost And Violation Over Iterations For Fixed Slopes

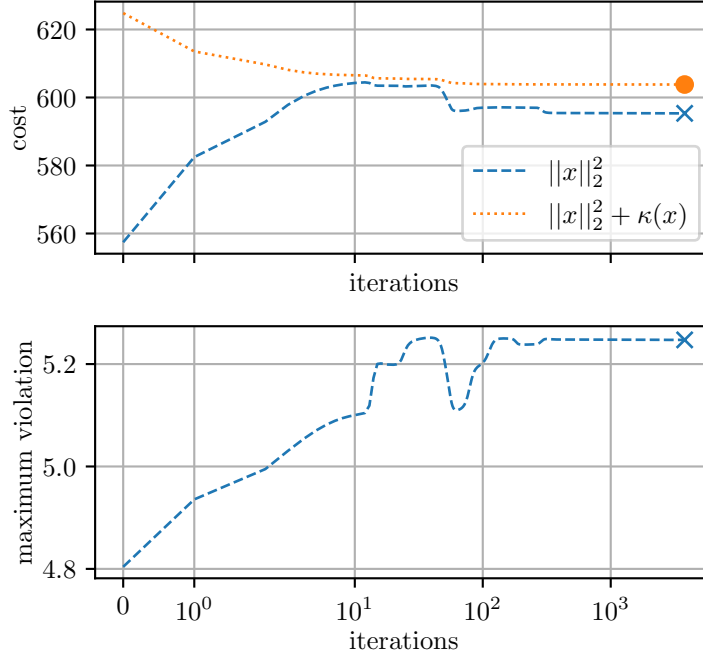


Figure 2.10: Violation and cost evolution over the iterations for fixed slopes  $\beta_i$ .

We can state the cost, which the factor graph actually should minimize as

$$Q(x) = \|x\|_2^2 + \kappa(x) \quad (2.77)$$

$$= \|x\|_2^2 + 2 \sum_{i=1}^n \beta_i \begin{cases} 0, & C_{i,\cdot} x \leq b_i, \\ C_{i,\cdot} x - b_i & C_{i,\cdot} x > b_i. \end{cases} \quad (2.78)$$

The actual cost is illustrated in orange. We observe that this cost remains non-increasing over the iterations, as expected from an iterative algorithm for convex problems.

In Figure 2.11 we depict the scaling of the two IFFBDD algorithms. Plotted is the median of a repeated experiment, where we have as many constraints as the dimensionality of  $x$ . We observe that although the method with  $\gamma = \infty$  requires many more iterations to converge, its speed in terms of milliseconds is orders of magnitude larger, particularly in higher dimensions. This agrees with the rough theoretical calculations in Section 2.3.2.

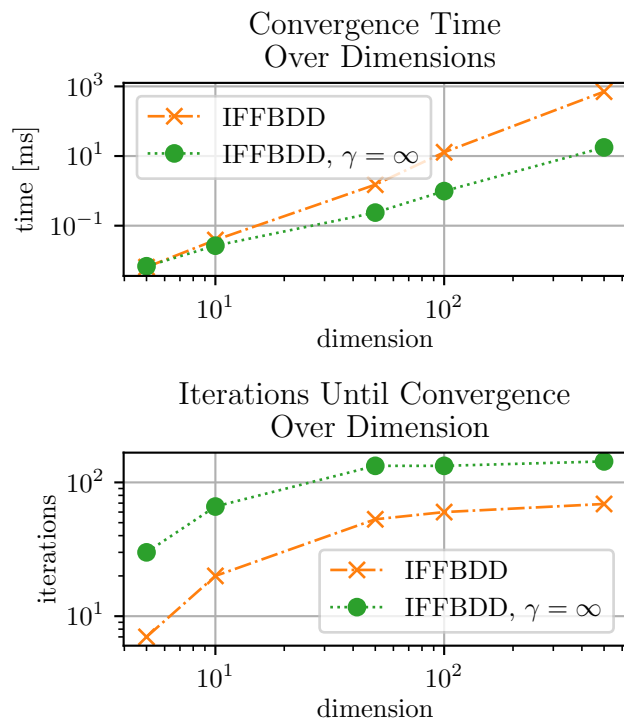


Figure 2.11: Convergence of the methods for increasing dimensions.

## Chapter 3

# State Space Models

Many systems evolve over time. In Chapter 2 we described "static" systems, which have no concept of time. State space models can capture this dynamic time behavior, and in this chapter, we investigate how to enforce constraints on the output of such systems.

We can extend Algorithm 2 to solve state space models given in the form of

$$\begin{aligned} X_\ell &= AX_{\ell-1} + BU_\ell \\ Y_\ell &= CX_\ell, \end{aligned} \tag{3.1}$$

with some fixed initial state  $X_0 = x_0$  and for  $\ell = 1, \dots, L$  time steps.

We display the corresponding factor graph in Figure 3.1, which illustrates a single time step of the state space model given by Equation (3.1). The input prior  $\text{NUP}_{U_\ell}$  is not a constraint, but it can model parts of the objective we want to minimize, such as the input energy or the absolute value of the input. We can include the constraints or other costs as tabulated in Table 2.1 and Table 2.2 in the output. We cannot place these dual NUP boxes on the input, because in the dual decision step, we have zero precision, which fixes the messages through the  $+$ -nodes; consider further [3, Equations 3.31 and 3.32]. This is analogous to a fixed message, i.e.,  $\vec{V}_X = 0$ , at an equality node, which fixes both other messages.

The difficulty of the algorithm to solve the state space model is not increased compared to the constrained optimization. We adapted this algorithm from [1, Algorithm 2], with simplified messages that only utilize primal Gaussian messages.

Note, compared to the constrained optimization, we cannot simplify the IFFBDD algorithm for state space models Algorithm 4 by letting  $\gamma \rightarrow \infty$ . The linear block  $A$  generally transforms the forward covariance  $\vec{V}_{X'_\ell}$  into a non-diagonal matrix. Thus, we gain nothing by using a first-order method to solve this type of problem, and we stick to the second-order method in the state space model.

**Algorithm 4** IFFBDD for state space model

---

```

1: Initialize  $\overleftarrow{m}_{Y_{\ell,i}} = 0$ ,  $\overleftarrow{V}_{Y_{\ell,i}} = 1$  and  $\gamma_{\ell,i} = 10^{-3}$  for all  $\ell, i$ 
2: while not converged do
3:   Standard forward Kalman filtering:
4:    $\vec{m}_{X_0} = x_0$  and  $\vec{V}_{X_0} = 0$ 
5:   for  $\ell = 1$  to  $L$  do
6:     Linear mapping and input block:
7:      $\vec{m}_{X'_\ell} = A\vec{m}_{X_{\ell-1}} + B\vec{m}_{U_\ell}$ 
8:      $\vec{V}_{X'_\ell} = A\vec{V}_{X_{\ell-1}}A^\top + B\vec{V}_{U_\ell}B^\top$ 
9:     Output block:
10:     $\vec{m}_{X_{\ell,1}} = \vec{m}_{X'}$ 
11:     $\vec{V}_{X_{\ell,1}} = \vec{V}_{X'}$ 
12:    for  $i = 1$  to  $n$  do
13:       $G_{\ell,i} = (\overleftarrow{V}_{Y_{\ell,i}} + C_{i,\cdot}\vec{V}_{X_{\ell,i}}C_{i,\cdot}^\top)^{-1}$ 
14:       $\vec{V}_{X_{\ell,i+1}} = \vec{V}_{X_{\ell,i}} - \vec{V}_{X_{\ell,i}}C_{i,\cdot}^\top G_{\ell,i}C_{i,\cdot}\vec{V}_{X_{\ell,i}}$ 
15:       $\vec{m}_{X_{\ell,i+1}} = \vec{m}_{X_{\ell,i}} + \vec{V}_{X_{\ell,i}}C_{i,\cdot}^\top G_{\ell,i}(\overleftarrow{m}_{Y_{\ell,i}} - C_{i,\cdot}\vec{m}_{X_{\ell,i}})$ 
16:    end for
17:     $\vec{m}_{X_\ell} = \vec{m}_{X_{\ell,n+1}}$ 
18:     $\vec{V}_{X_\ell} = \vec{V}_{X_{\ell,n+1}}$ 
19:  end for
20:  Backward dual deciding:
21:   $\hat{\hat{x}}_T = 0$ 
22:  for  $\ell = T$  to  $1$  do
23:     $\hat{\hat{x}}_{\ell,n+1} = \hat{\hat{x}}_\ell$ 
24:    for  $i = n$  to  $1$  do
25:       $\vec{V}_{Y_{\ell,i}} = C_{i,\cdot}\vec{V}_{X_{\ell,i}}C_{i,\cdot}^\top$ 
26:       $\vec{m}_{Y_{\ell,i}} = C_{i,\cdot}(\vec{m}_{X_{\ell,i}} - \vec{V}_{X_{\ell,i}}\hat{\hat{x}}_{\ell,i+1})$ 
27:      Set  $\gamma_{\ell,i}$  and decide  $\hat{y}_{\ell,i}$  according to Table 2.1 and Table 2.2.
28:      Set  $\overleftarrow{m}_{Y_{\ell,i}}$ ,  $\overleftarrow{V}_{Y_{\ell,i}}$  according to Table 2.1 and Table 2.2.
29:       $\hat{\hat{x}}_{\ell,i} = \hat{\hat{x}}_{\ell,i+1} + C_{i,\cdot}^\top \hat{y}_{\ell,i}$ 
30:    end for
31:     $\hat{\hat{x}}'_\ell = \hat{\hat{x}}_{\ell,1}$ 
32:    Estimate inputs:
33:     $\hat{u}_\ell = \vec{m}_{U_\ell} - \vec{V}_{U_\ell}\hat{\hat{x}}'_\ell$ 
34:     $\hat{\hat{x}}_{\ell-1} = A^\top \hat{\hat{x}}'_\ell$ 
35:  end for
36:  Update  $\vec{\mu}_{U_\ell}$  according to the modeled prior.
37: end while

```

---

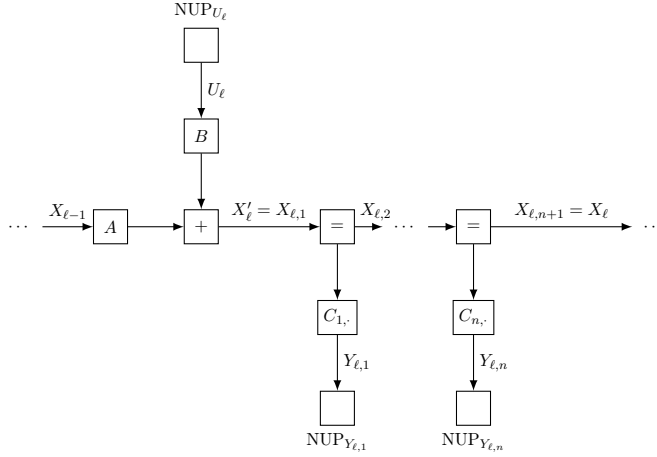


Figure 3.1: Factor graph of time step of time-invariant state space model.

Although we will restrict ourselves to linear time-invariant state space models in this project, this is not necessary. We could easily extend our algorithm to time-varying systems by making the matrices  $A$ ,  $B$ , and  $C$  time-dependent, i.e.,  $A_\ell$ ,  $B_\ell$ , and  $C_\ell$ .

### 3.1 Enforcing Constraints on Input

As explained in the preceding section, the dual decision mechanism does not offer to impose constraints on the input out of the box. However, we can modify our pertinent state space model to allow constraints on the current inputs. For a given state space model as in Equation (3.1), consider the modified state and output

$$\check{X} = \begin{bmatrix} X \\ U \end{bmatrix}, \check{Y} = \begin{bmatrix} Y \\ U \end{bmatrix}, \quad (3.2)$$

and thus the adapted model

$$\check{X}_\ell = \begin{bmatrix} X_\ell \\ U_\ell \end{bmatrix} \quad (3.3)$$

$$= \begin{bmatrix} AX_{\ell-1} + BU_\ell \\ U_\ell \end{bmatrix} \quad (3.4)$$

$$= \begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} X_{\ell-1} \\ U_{\ell-1} \end{bmatrix} + \begin{bmatrix} B \\ I \end{bmatrix} U_\ell \quad (3.5)$$

$$= \check{A}\check{X}_{\ell-1} + \check{B}U_\ell \quad (3.6)$$

$$(3.7)$$

and

$$\check{Y}_\ell = \begin{bmatrix} Y_\ell \\ U_\ell \end{bmatrix} \quad (3.8)$$

$$= \begin{bmatrix} CX_\ell \\ U_\ell \end{bmatrix} \quad (3.9)$$

$$= \begin{bmatrix} C & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} X_\ell \\ U_\ell \end{bmatrix} \quad (3.10)$$

$$= \check{C}\check{X}_\ell. \quad (3.11)$$

Now we can put costs and constraints on the inputs by adding them to the outputs of the modified state space model. However, we need to take into account that we have a quadratic scaling in terms of the dimension of  $x$ , as discussed in Section 2.3.2. For larger state space models, we may observe a significant decrease in convergence speed.

## 3.2 Composite NUP Priors

In the context of a state space model, we may encounter a scenario where we want to assign different costs and constraints to the same output. For example, a race car should reach the end of a track as quickly as possible, but it must not overshoot the finish line. Then the incentive for the car to reach the finish line can be modeled by a cost, and a constraint can ensure that it does not overshoot at the end. We could implement this by duplicating the desired output row  $C_{i,\cdot}$  of the observation matrix and putting the cost on one observation and the constraint on the duplicated one. However, if the state space model is large in terms of the dimension of  $X$  or if we have a large number of time steps  $L$ , this is a costly operation.

Another approach, as done here [2], is to use composite NUP priors. We can use a chain of equalities with different NUPs to model different constraints in the constrained optimization Equation (2.1). This allows us to model costs and constraints on the scalar observation  $Y_{\ell,i} = C_{i,\cdot}X_\ell$ . Consider further Figure 3.2.

We get the box  $\vec{\mu}_{Y_{\ell,i}}$  as

$$\vec{V}_{Y_{\ell,i}} = C_{i,\cdot} \vec{V}_{X_{\ell,i}} C_{i,\cdot}^\top \quad (3.12)$$

$$\vec{m}_{Y_{\ell,i}} = C_{i,\cdot} (\vec{m}_{X_{\ell,i}} - \vec{V}_{X_{\ell,i}} \hat{x}_{\ell,i+1}) \quad (3.13)$$

with mean and covariance taken from Algorithm 4, Line 25 and Line 26. Then, done differently than in [2], we can perform scalar forward filtering and backward dual deciding in this composite NUP prior.

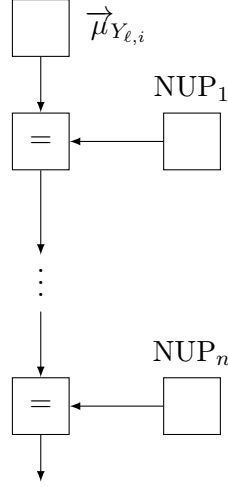


Figure 3.2: Factor graph to model composite NUP prior.

There is a pitfall in dual deciding a composite NUP prior: We might be tempted to do one forward filtering and one backward dual deciding step, and then continue to dual decide the factor graph of the state space model. However, a dual decision corresponds to

$$\hat{\tilde{y}} = \underset{\tilde{y}}{\operatorname{argmin}} \tilde{f}(\tilde{y}), \quad (3.14)$$

where  $\tilde{f}$  is the objective function for that composite NUP. Consider further [1, Section III.D]. One FFBDD step does not suffice to minimize Equation (3.14), because the NUPs represent the dual costs iteratively. Thus, we need to wait until the *inner* IFFBDD algorithm converges before propagating this dual-decision scalar back into Algorithm 4. We create the next Gaussian message for the forward filtering step by collecting the Gaussian messages from the individual NUPs in the composite structure after convergence. We encourage the reader to use  $\vec{\xi}_Y$  and  $\vec{W}_Y$  as the message representation in the composite structure. Collecting the individual messages is then simply a sum. Using the provided costs and constraints, as shown in Table 2.1 and Table 2.2, we can model custom costs and constraints by combining the building blocks in composite NUP priors.



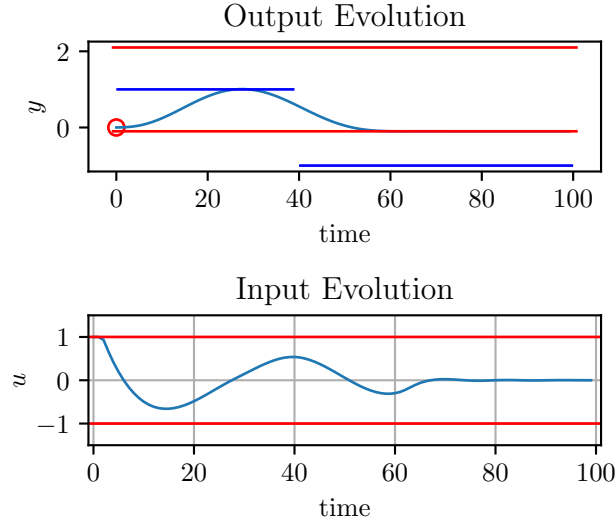


Figure 3.3: Solved state space model with input and output constraints and composite NUPs.

### 3.3 Example And Limitations

Combining the methods above allows us to present an example. We chose the dynamics of the state space model to be

$$X_\ell = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1/2 & 1 & 1 \end{bmatrix} X_{\ell-1} + s \begin{bmatrix} 1 \\ 1/2 \\ 1/3 \end{bmatrix} U_\ell, \quad (3.15)$$

$$Y_\ell = [0 \quad 0 \quad 1] X_\ell, \quad (3.16)$$

where  $s$  is a small constant. We formulate our objective as:

$$\text{minimize } \sum_{\ell} (U_\ell^2 + \beta |Y_\ell - \check{y}_\ell|), \quad (3.17)$$

$$\text{subject to linear constraints.} \quad (3.18)$$

We want to minimize the input energy  $U_\ell^2$ , and our output should follow the trajectory  $\check{y}_\ell$ . We depict the solved state space model in Figure 3.3. The red lines indicate the constraints. Note that we require our state space model to start at the red circle. The dark blue represents the trajectory the output should follow, and the light blue line depicts the evolution of the state space model. We observe expected behavior. The output  $Y_\ell$  follows the trajectory  $\check{y}_\ell$  while ensuring that the constraints are always satisfied. By tweaking the slope parameter  $\beta$  in Equation (3.17), we can manage the tradeoff between minimizing the input energy and the output following the trajectory.

However, numerical experiments revealed several limitations; we will discuss a few of them next.

1. Compared to Section 2.2.4, where we set  $\gamma = 0$ , we cannot easily initialize our interval parameters  $\gamma_{\ell,i}$  with zero. Doing so in experiments leads to a degenerate forward covariance matrix  $\vec{V}_{X_{\ell,i}}$  and results in

$$C_{i,\cdot} \vec{V}_{X_{\ell,i}} C_{i,\cdot}^\top = 0. \quad (3.19)$$

This resulted in 0/0, which the implemented algorithm was unable to recover from. Consider further Equation (2.43). Further investigation revealed that we often have the  $A$ ,  $B$ , and  $C$  matrices in the state space model, such that the rank-one update in Line 14 of Algorithm 4 perfectly cancels a row and column of the forward covariance matrix  $\vec{V}_{X_{\ell,i}}$ , which explains Equation (3.19). In the state space model setting, we thus needed to resort to initializing the  $\gamma$  with a small value, e.g.,  $10^{-3}$ . Consequently, we could not use the equality constraint as described in Table 2.4; instead, we used an interval constraint and set the border  $a = b = c$ .

Still, we believe that we should be able to initialize the constraints with  $\gamma = 0$ . If we then encounter the problem of Equation (3.19), we can interpret this as follows: the state is fixed at this time step. Thus, as long as this fixed state does not violate any constraints now or in later time steps, we should be able to continue with our algorithm just fine. However, we did not implement this relaxation, and it remains to be tested.

2. We initialize the backward messages  $\overleftarrow{\xi}_{Y_{\ell,i}}$  and  $\overleftarrow{W}_{Y_{\ell,i}}$  as:
  - Constraints:  $\overleftarrow{\xi}_{Y_{\ell,i}}$  at or between the borders and  $\overleftarrow{W}_{Y_{\ell,i}} = 1$ .
  - Costs:  $\overleftarrow{\xi}_{Y_{\ell,i}}$  at or the borders and  $\overleftarrow{W}_{Y_{\ell,i}} = 0$ .

However, when observing the evolution of the state space model over the iterations of the IFFBDD algorithm, we noticed a rapid increase in the magnitude of the  $\gamma$  in the first few iterations. This is problematic because, for large  $\gamma$ , the second-order method begins to resemble a first-order method, which is slower in terms of the number of iterations required for convergence compared to the second-order method. We noticed this slow convergence when solving objectives of state space models subject to linear constraints. The example in Equation (3.17) took many thousands iterations until convergence.

It remains to be tested whether we could speed up convergence by a smart initialization of the backward messages, such that the magnitude of the  $\gamma$  does not explode in the first iterations.

3. Composite NUP priors allow for great flexibility. However, when we have many composite NUPs in our setup, the speed loss is noticeable because

building up this inner IFFBDD loop and waiting for convergence takes much longer than a tabulated rule. It would be great if we could derive tabulated update rules for composite NUP priors, as this would allow us to skip the inner loop. So we want to derive a closed-form update rule for a dual representation of a composite cost  $\kappa_{\text{comp}}$ , where

$$\kappa_{\text{comp}}(y) = \sum_j \kappa_j(y), \tag{3.20}$$

subject to  $y$  satisfying constraints.

In the end, we did not have sufficient time to compare Algorithm 4 with an adapted version of Algorithm 1 or other solvers. It remains to be tested how Algorithm 4 performs compared to available solvers for state space models.

## Chapter 4

# Implementation

Part of this project involved learning the Rust programming language alongside the theoretical material. The Rust programming language has recently gained popularity due to some key promises it makes. With its concept of *ownership and borrowing*, Rust is a memory-safe Language with no garbage collection. This concept offers great performance promises, which we desire for our algorithms, as they can grow to substantial computational loads. A great source for learning Rust is provided here [9]. We provide all the source code for the described algorithms under [https://github.com/flocke992/master\\_project](https://github.com/flocke992/master_project).

### 4.1 General Structure

For both IFFBDD algorithms — the one that performs constrained optimization, as described in Equation (2.1), and the one that solves state space models under constraints — we have a similar structure, and both are contained within their respective modules. In the respective `mod`’s we have helper `struct`’s and `enum`’s to produce custom errors, set prior objectives, such as  $L_2$  or  $L_1$  cost, and permit easier problem instantiations. The general structure of both algorithms is that there is a top-level `struct` that handles the logic of the algorithm, i.e., resetting forward messages, initializing the dual deciding phase, passing these messages from one block to the next in correct order, handling the correct prior, and checking for convergence. The blocks, namely `LinearBlock`, `InputBlock`, and `OutputBlock`, then internally implement their logic of the `forward_filter()` and `backward_decide()` methods. This allows the top-level structure to call the required `forward_filter()` or `backward_decide()` incrementally, creating more readable code. The constrained optimizer, as specified in Equation (2.1), only requires the `OutputBlock`. For the state space model, we inserted an additional abstraction layer, namely, a `ModelStep`, which models a single time step in the discrete-time LTI system. Then the state space model solver holds all the `ModelStep`’s, which internally hold the respective blocks.

```
enum Sense {
    Less(f64),
    Greater(f64),
    Equal(f64),
    Interval(f64, f64),
    L1(f64, f64),
    Hinge1(f64, f64),
    Hinge2(f64, f64),
    Vapnik(f64, f64, f64),
}
```

Figure 4.1: Enumeration to model first-order NUP.

We tried to minimize memory allocations during runtime. By allocating the necessary memory when instantiating the structures and then reusing these parts, we achieved these reduced memory allocations. For example, we can reuse the allocated memory for the mean message  $\vec{m}_{X_i}$  as accumulated dual decided message  $\hat{x}_i$ . Additionally, the implementation of the NUP boxes requires to store values, e.g., the backward message  $\overleftarrow{\mu}_{Y_i}$  for the next Kalman filtering step, which is updated in the decision phase. Whenever possible, we attempted to reuse fields for different use cases while maintaining code readability. One might argue that one `f64` field more or less in a structure is not that important; however, consider the following example:

For a state space model with  $L = 150$  time steps, and in each time step we have  $n = 6$  constraints or costs, we get  $150 \cdot 6 \cdot 8 = 7.2\text{kB}$ . The program will simply drag this memory along in cache. This example could be a state-space model that models a 2D car with constraints on the input acceleration, a cost function, and a constraint on the car's position. Even in this small example, one `f64` field more or less leads to a notable difference in memory usage.

## 4.2 Enumerations

Enumerations (enums) are an excellent fit for modeling the different costs and constraints in Algorithm 4. Enumerations allow us to define a type by enumerating its possible variants; an enum can only be one concrete variant, and conveniently, the variants can store data [9, Chapter 6].

In the implementation, a first-order NUP box contains a field called *sense* that models costs and constraints as in Figure 4.1

Then further down in the program flow, we can make a *dual decision* based on the concrete variant the enum represents. We provide a code snippet of such a decision in Figure 4.2.

```

fn dual_decide(mean: f64, var: f64, sense: Sense) -> f64 {
  match sense {
    Sense::Less(b) => {
      ((mean - b) / var).max(0.0)
    }
    Sense::Greater(a) => {
      ((mean - a) / var).min(0.0)
    }
    Sense::Equal(c) => {
      (mean - c) / var
    }
    Sense::Interval(a, b) => {
      ((mean - a) / var).min(0.0)
      + ((mean - b) / var).max(0.0)
    }
    Sense::L1(beta, c) => {
      ((mean - c) / var).clamp(-beta, beta)
    }
    Sense::Hinge1(beta, a) => {
      ((mean - a) / var).clamp(-beta, 0.0)
    }
    Sense::Hinge2(beta, b) => {
      ((mean - b) / var).clamp(0.0, beta)
    }
    Sense::Vapnik(beta, a, b) => {
      ((mean - a) / var).clamp(-2.0 * beta, 0.0)
      + ((mean - b) / var).clamp(0.0, 2.0 * beta)
    }
  }
}

```

Figure 4.2: Dual decide first-order NUP

We can understand the *match* statement as a switch/case statement in other programming languages. Since the algorithm’s structure will be known at compile-time, the compiler will be able to insert the concrete match arm into the flow of the program, and the compiler can discard the others for that particular NUP. This results in this part of the program being branchless, which is excellent for execution speed.

### 4.3 Reference Counting

Rust introduced a concept called *ownership*, which allowed it to be memory-safe without needing a garbage collector. By enforcing the following rules, memory allocations and deallocations can be managed by the program itself [9, Chapter 4].

- Each value in Rust has an owner.
- There can only be one owner at a time.
- When the owner goes out of scope, the program drops the value.

Moreover, similar to other languages, we can also have references to values, which do not transfer ownership. However, we must ensure that the owner of the value does not go out of scope while there is still a reference to that value [9, Chapter 10].

Each block of the implementation of the IFFBDD algorithm for the state space model in Figure 3.1 requires mapping Gaussian messages via a linear transformation. These matrices need to be owned to be allocated, and it makes sense that each block owns its corresponding linear mappings. However, for time-invariant state space models, this would create numerous copies of the same data structure, which is wasteful. Instantiating the matrices at a central location and providing references to the individual blocks would work. However, this makes the code unreadable and may be tedious. For this problem, Rust offers a structure called a [Reference counter](#). This structure allows shared ownership of an internal value. We depict the usage of this `Rc` in our implementation in Figure 4.3. Note that when creating a new `InputBlock`, the struct owns the `b` field, and cloning an `Rc` does not copy all the internal values; instead, it produces a new pointer to the same allocation. Conveniently, `Rc` automatically dereferences the data inside and can thus be used just like a normal `Vec`. With this implementation, each `InputBlock` owns its own linear mapping and eliminates duplicated allocations of the same matrices.

### 4.4 BLAS

All the different algorithms used throughout the project perform numerous linear algebra operations, e.g.  $C_i \cdot \vec{V}_{X_i} C_i^\top$ , or  $A \vec{V}_{X_i} A^\top$ . Whenever linear al-

```

use std::rc::Rc;
impl InputBlock {
    fn new(b_ref: &Rc<[f64]>, ...) -> Self {
        ...
        InputBlock {
            b: Rc::clone(b_ref),
            ...,
        }
    }
    fn forward_filter(
        &self, m_x: &mut [f64], v_x: &mut [f64]
    ) {...}
    fn backward_decide(&mut self, dual_x_hat: &[f64]) {...}
}

```

Figure 4.3: Reference counting matrices.

gebra operations were required, we utilized a *Basic Linear Algebra Subprogram* (BLAS) as the backend. These provide a set of low-level routines for performing linear algebra operations, such as vector operations, matrix-vector operations, and matrix-matrix operations. Using BLAS offers near state-of-the-art performance for these types of operations, as it performs extensive optimization, far beyond the scope of this project. We illustrate a typical BLAS call in Figure 4.4. We can note a few things. Firstly, all BLAS calls are unsafe. We are using `cblas`, which is the C implementation of BLAS. Calls to foreign functions that are not written in Rust are unsafe. To invoke these BLAS functions, we need to wrap them in an *unsafe* block. If we ensure the correct dimensions, we achieve greater speed because Rust will not perform index checks on the slices passed to the BLAS calls. During implementation, we ensure that we use correct dimensions when instantiating the corresponding structure, such as an input block. Secondly, we notice that the BLAS calls are challenging to read. Thus, for more complex blas calls, i.e., a general matrix-matrix multiplication, we recommend using a few unit tests to ensure the correct method call. A nice overview of the different BLAS calls is provided in the [Quick Reference Guide](#). The signatures of the BLAS calls often allow for scaling in the updating step, i.e.,  $y = y + \alpha x$ . When possible, we attempted to incorporate these scaling steps into the IFFBDD algorithm to minimize redundant operations. To compute the quantity  $A\vec{V}_{X_i}A^\top$  in BLAS, we need to store the intermediate result  $A\vec{V}_{X_i}$ , but if we allocate such an intermediate space in each linear block, we waste much memory. To provide scratch space for each linear block to write intermediate results, a pattern with interior mutability was used [9, Chapter 15]. Each linear block holds a `scratch_space` field, which is of type `Rc<RefCell<[f64]>>`. The `Rc` ensures that only one such structure exists, even though there are many linear



```

use cblas::daxpy;
unsafe{
    //update y = y + alpha*x
    //x,y are vectors, alpha is a scalar
    //dim(y)=k,
    daxpy(
        k, //dim
        alpha, //scaler
        &x, //first vector
        1, //stride for first vector
        &mut y, //vector to update
        1 //stride for second vector
    );
}

```

Figure 4.4: Exemplary Rust blas call.

blocks, and `RefCell` allows interior mutability for an immutable object. See also: [Module cell](#).

## 4.5 Traits

Although Algorithm 3 and Algorithm 2 solve the same objective, their internal implementations differ. Nevertheless, the end-user needs no knowledge of the various implementations. He needs some exposed methods that he can call to set up the problem as given in Equation (2.1) and solve it. However, if he wants to maintain flexibility and choose the type he ultimately uses, we can employ traits as bounds.

A trait has the following characteristics:

- A trait defines the functionality a type has and can share with other types [9, Chapter 10]. In this example, the shared behavior is that both methods can solve a constrained optimization problem.
- Traits are similar to interfaces, but with static dispatch. We can then create a method signature that expects a trait as a parameter. At compile time, the compiler inserts the concrete type, i.e., static dispatch; this is a zero-cost abstraction feature.

We present the trait structure for the trait `Solver` in Figure 4.5. We can think of traits as defining the code skeleton and explaining the behavior of the trait, while the concrete implementation of the struct handles the logic of the trait.

We could also implement the different costs and constraints by defining a trait that specifies the signature of the `forward_filter` and `backward_decide`

```

trait Solver {
    /// Adds a constraint to the instantiated solver.
    fn add_constraint(&mut self, constraint: Sense);
    /// Minimizes objective under linear constraints.
    fn solve(&mut self, objective: Objective) ->
        Result<Vec<f64>, SolveError>;
    /// Checks the maximum violation for the candidate.
    fn check_constraints(&self, x_hat: &[f64]) -> f64;
}

```

Figure 4.5: Type definition of constrained optimization solver.

methods for their respective behaviors. Then we could define an array that stores structs that implement this trait.

This works, but because arrays require types that all have the same size, and this is not guaranteed in this case, the concrete types are erased, and the array stores pointers to a generated vtable, which points to the concrete types. Consider further [dynamic dispatch](#) and [vtable](#). However, because our costs and constraints will be invoked many times over the iterations, having such a layer of abstraction results in the program needing to perform a lookup every time we want to apply logic according to a particular constraint. Because constraints are on a very hot path, this approach might be notably slower. A hot path in a program is a code snippet that gets invoked many times. So we used enums to model constraints. Although we have utilized traits in this project, their application was limited. Nevertheless, they encouraged similar method signatures between the solvers, which is good for readability and maintainability. Traits are more useful for an end-user who wants to use these solvers in a larger project.

## 4.6 Learnings

Throughout the project, we began to value Rust as a programming language that offers maintainable code bases through integrated unit tests and a compiler that performs extensive checking.

However, the compiler is strict. For example, our code does not compile until we handle all variants of an enum in a `match` statement. Due to the extensive safety checking by the compiler, prototyping was tedious. Often, we wanted to test a quick feature that affected a small portion of the code-base. However, we needed to modify the entire code to implement this minor feature, as otherwise, the compiler would not be satisfied. Additionally, the code sometimes suffered from premature optimization, where we attempted to implement near-optimal code before we developed the complete code skeleton. This slowed down prototyping. We could have expected to need time to refactor our implementations eventually. Taking this into account would have

allowed a faster first implementation. If we had planned for a refactoring and optimization time slot in the end, there could have been better code quality and performance.

## Chapter 5

# Conclusions And Future Work

We have successfully implemented three constrained optimization algorithms, simplified some expressions from [1], and compared these algorithms in various ways. Among other things, we have roughly analyzed the different algorithms in terms of computational complexity and memory usage, and studied how two prior methods to minimize the  $L_1$  cost behave. After that, we adapted Algorithm 2 to the state space model setting, modified the state space model to accommodate constraints on the input, and worked out an approach to handle composite costs in the dual NUP representation setting. In the end, we highlighted a few key aspects of the Rust implementation of these algorithms, particularly reference counting in Section 4.3, which allowed us to harness the safety features of Rust while maintaining a slim resource footprint.

While we have explored some parts of the presented algorithms, we could continue our work in different directions listed below:

1. Check convergence speed by comparing Vapnik/Interval cost with a single  $\gamma$  parameter as in [1, Appendix A] or as presented here in this report with two  $\gamma$ .
2. Check convergence speed by comparing the  $L_1$  prior methods for Algorithm 2 mentioned in Section 2.3.1.
3. Can we have faster convergence in the state space model with smart initialization of NUP parameters?
4. Can we implement composite NUPs with closed-form solutions?
5. Compare the performance of Algorithm 4 to other state space model solvers.
6. Optimize the Rust implementation further by using a [profiler](#) to measure performance and locate performance bottlenecks.
7. Work out the rules in the state space model to work with the  $\gamma$  initialized with 0.

8. Extend the state space model implementation to linear time-varying models.

# Bibliography

- [1] Y.-P. Li and H.-A. Loeliger, “Dual nup representations and minimization in factor graphs,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.12113>
- [2] R. Keusch and H.-A. Loeliger, “Model-predictive control with nup priors,” 2025. [Online]. Available: <https://arxiv.org/abs/2303.15806>
- [3] H. A. Loeliger, “Model-based estimation and signal analysis,” 2024.
- [4] Y.-P. Li and H.-A. Loeliger, “Backward filtering forward deciding in linear non-Gaussian state space models,” in *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, S. Dasgupta, S. Mandt, and Y. Li, Eds., vol. 238. PMLR, 02–04 May 2024, pp. 2287–2295. [Online]. Available: <https://proceedings.mlr.press/v238/li24j.html>
- [5] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [6] J. Sherman and W. J. Morrison, “Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix,” *The Annals of Mathematical Statistics*, vol. 21, no. 1, pp. 124 – 127, 1950. [Online]. Available: <https://doi.org/10.1214/aoms/1177729893>
- [7] C. Hildreth, “A quadratic programming procedure,” *Naval Research Logistics Quarterly*, vol. 4, no. 1, pp. 79–85, 1957. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800040113>
- [8] L. Gurobi Optimization, “Gurobi optimizer reference manual,” 2024. [Online]. Available: <https://www.gurobi.com>
- [9] S. Klabnik, C. Nichols, C. Krycho, and T. R. Community, *The Rust Programming Language*, 2nd ed. No Starch Press, 2023. [Online]. Available: <https://doc.rust-lang.org/stable/book/>

# List of Figures

|      |                                                                                                                                                                                        |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Visualization of the hinge II cost function. . . . .                                                                                                                                   | 5  |
| 2.2  | Factor graph to model constrained optimization. . . . .                                                                                                                                | 5  |
| 2.3  | Dual deciding a NUP. . . . .                                                                                                                                                           | 8  |
| 2.4  | Update rules for combined NUP to model interval constraint. .                                                                                                                          | 10 |
| 2.5  | IFFBDD in backward dual deciding phase. . . . .                                                                                                                                        | 14 |
| 2.6  | Factor graph of $L_1$ objective prior box with dual $L_1$ costs for Algorithm 3. . . . .                                                                                               | 21 |
| 2.7  | Convergence speed of different $L_1$ prior methods. The distribution of convergence time is depicted in light blue. The blue bar indicates the median. . . . .                         | 22 |
| 2.8  | Convergence speed for repeated experiment with $k = 50$ and 50 constraints. The distribution of convergence time is depicted in light blue. The blue bar indicates the median. . . . . | 27 |
| 2.9  | Violation and cost evolution over the iterations of the algorithms. The experiment was $k = 100$ and 100 constraints. . . .                                                            | 27 |
| 2.10 | Violation and cost evolution over the iterations for fixed slopes $\beta_i$ . . . . .                                                                                                  | 28 |
| 2.11 | Convergence of the methods for increasing dimensions. . . . .                                                                                                                          | 29 |
| 3.1  | Factor graph of time step of time-invariant state space model. .                                                                                                                       | 32 |
| 3.2  | Factor graph to model composite NUP prior. . . . .                                                                                                                                     | 34 |
| 3.3  | Solved state space model with input and output constraints and composite NUPs. . . . .                                                                                                 | 35 |
| 4.1  | Enumeration to model first-order NUP. . . . .                                                                                                                                          | 39 |
| 4.2  | Dual decide first-order NUP . . . . .                                                                                                                                                  | 40 |
| 4.3  | Reference counting matrices. . . . .                                                                                                                                                   | 42 |
| 4.4  | Exemplary Rust blas call. . . . .                                                                                                                                                      | 43 |
| 4.5  | Type definition of constrained optimization solver. . . . .                                                                                                                            | 44 |

# List of Tables

|     |                                                                                                                                             |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Dual variable update rules for all dual costs with manual $\gamma$ . . .                                                                    | 9  |
| 2.2 | Summarized update rules for all dual constraints. . . . .                                                                                   | 11 |
| 2.3 | The $L_1$ cost function $\kappa$ , its dual $\kappa^*$ and its dual proxy $\check{\kappa}$ taken<br>from [1, Table I and Table II]. . . . . | 13 |
| 2.4 | Update rules for equality constraint with $\gamma = 0$ . . . . .                                                                            | 16 |
| 2.5 | Comparison of equality constraint methods for repeated exper-<br>iment with 100 equality constraints and $x \in \mathbb{R}^{100}$ . . . . . | 16 |
| 2.6 | Update rules for dual costs with $\gamma \rightarrow \infty$ . . . . .                                                                      | 17 |
| 2.7 | Update rules for constraints with $\gamma \rightarrow \infty$ . . . . .                                                                     | 18 |



# Declaration Of Use Of AI-Based Tools

| AI-Based Tool                                   | Use Case                                                   | Scope         | Remarks                                                            |
|-------------------------------------------------|------------------------------------------------------------|---------------|--------------------------------------------------------------------|
| Grammarly<br>Last date of<br>access: 09.01.2025 | Correction of<br>grammatical<br>and stylistic<br>mistakes. | Entire report | Premium plan<br>with Overleaf<br>integration.                      |
| ChatGPT<br>Last date of<br>access: 09.01.2025   | Search engine<br>replacement                               | Entire work   | For example, a<br>sanity check in a<br>mathematical<br>expression. |
| ChatGPT<br>Last date of<br>access: 09.01.2025   | Debugging tool<br>and code<br>exploration                  | Rust program  | Mainly used as<br>a search engine<br>replacement.                  |



Eidgenössische Technische Hochschule Zürich  
Swiss Federal Institute of Technology Zurich

## Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

**Title of work** (in block letters):

On Constrained Convex Optimization Via Dual NUP Representations

**Authored by** (in block letters):

*For papers written by groups the names of all authors are required.*

**Name(s):**

Achermann

**First name(s):**

Cyrill Jan

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

**Place, date**

Zürich, 01.09.2025

**Signature(s)**

C. Achermann

*For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.*